



Universidade Estadual de Campinas

Enemy Leo used Appeal! It's
super effective! Pikachu fainted!

Pedro Assunção, Pedro Ferreira, Yvens Porto

2026-08-25

1	Contest	1
2	Mathematics	1
2.1	Equations	1
2.2	Recurrences	1
2.3	Trigonometry	1
2.4	Geometry	1
2.5	Derivatives/Integrals	2
2.6	Sums	2
2.7	Series	2
2.8	Probability theory	2
2.9	Markov chains	3
3	Data structures	3
4	Numerical	5
4.1	Polynomials and recurrences	5
4.2	Optimization	6
4.3	Matrices	7
4.4	Fourier transforms	8
5	Number theory	9
5.1	Modular arithmetic	9
5.2	Primality	9
5.3	Divisibility	10
5.4	Fractions	10
5.5	Pythagorean Triples	11
5.6	Primes	11
5.7	Estimates	11
5.8	Mobius Function	11
6	Combinatorial	11
6.1	Permutations	11
6.2	Partitions and subsets	11
6.3	General purpose numbers	11
7	Graph	12
7.1	Fundamentals	12
7.2	Network flow	12
7.3	Matching	14
7.4	DFS algorithms	14
7.5	Coloring	15
7.6	Heuristics	16
7.7	Trees	16
7.8	Math	18
8	Geometry	18
8.1	Geometric primitives	18
8.2	Circles	19
8.3	Polygons	20
8.4	Misc. Point Set Problems	21

8.5	3D	23
9	Strings	23
10	Various	25
10.1	Intervals	25
10.2	Misc. algorithms	25
10.3	Dynamic programming	26
10.4	Debugging tricks	26
10.5	Optimization tricks	26
Contest (1)		
template.cpp	14 lines	
#include <bits/stdc++.h> using namespace std; #define rep(i, a, b) for(int i = a; i < (b); ++i) #define all(x) begin(x), end(x) #define sz(x) (int)(x).size() typedef long long ll; typedef pair<int, int> pii; typedef vector<int> vi; int main() { cin.tie(0)->sync_with_stdio(0); cin.exceptions(cin.failbit); }		
.bashrc	3 lines	
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \ -fsanitize=undefined,address' xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps =< .vimrc		
6 lines set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul sy on im jk <esc> im kj <esc> no ; : " Select region and then type :Hash to hash your selection. " Useful for verifying that there aren't mistypes. ca Hash w !cpp -dD -P -fpreprocessed \ tr -d '[:space:]' \ \ md5sum \ cut -c-6		
hash.sh	3 lines	
# Hashes a file, ignoring all whitespace and comments. Use for # verifying that code was correctly typed. cpp -dD -P -fpreprocessed \ tr -d '[:space:]' md5sum cut -c-6		
hash2.sh	2 lines	
# bash hash.sh name.cpp l1 l2 (hash of lines [l1, l2]): sed -n \$2','\$3' p' \$1 sed '/^#w/d' cpp -dD -P - fpreprocessed \ tr -d '[:space:]' md5sum cut -c-6		
hashCheck.h		
Description: Use this for checking the hash.sh. 8827b8, 4 lines ../number-theory/ModPow.h // Computes modular inverse. Assumes mod is a prime. a26 ll modinv(ll n) { 033 return modpow(n, mod - 2); 882 }		

troubleshoot.txt	52 lines
Pre-submit: Write a few simple test cases if sample is not enough. Are time limits close? If so, generate max cases. Is the memory usage fine? Could anything overflow? Make sure to submit the right file. Wrong answer: Print your solution! Print debug output, as well. Are you clearing all data structures between test cases? Can your algorithm handle the whole range of input? Read the full problem statement again. Do you handle all corner cases correctly? Have you understood the problem correctly? Any uninitialized variables? Any overflows? Confusing N and M, i and j, etc.? Are you sure your algorithm works? What special cases have you not thought of? Are you sure the STL functions you use work as you think? Add some assertions, maybe resubmit. Create some testcases to run your algorithm on. Go through the algorithm for a simple case. Go through this list again. Explain your algorithm to a teammate. Ask the teammate to look at your code. Go for a small walk, e.g. to the toilet. Is your output format correct? (including whitespace) Rewrite your solution from the start or let a teammate do it. Runtime error: Have you tested all corner cases locally? Any uninitialized variables? Are you reading or writing outside the range of any vector? Any assertions that might fail? Any possible division by 0? (mod 0 for example) Any possible infinite recursion? Invalidated pointers or iterators? Are you using too much memory? Debug with resubmits (e.g. remapped signals, see Various). Time limit exceeded: Do you have any possible infinite loops? What is the complexity of your algorithm? Are you copying a lot of unnecessary data? (References) How big is the input and output? (consider scanf) Avoid vector, map. (use arrays/unordered_map) What do your teammates think about your algorithm? Memory limit exceeded: What is the max amount of memory your algorithm should need? Are you clearing all data structures between test cases?	

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n = c_1a_{n-1} + \dots + c_ka_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1r_1^n + \dots + d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1n + d_2)r^n$.

2.3 Trigonometry

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \end{aligned}$$

$$\begin{aligned} \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c

$$\text{Semiperimeter: } p = \frac{a + b + c}{2}$$

$$\text{Area: } A = \sqrt{p(p - a)(p - b)(p - c)}$$

$$\text{Circumradius: } R = \frac{abc}{4A}$$

$$\text{Inradius: } r = \frac{A}{p}$$

Length of median (divides triangle into two equal-area triangles):
 $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$$

$$\text{Law of sines: } \frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$$

$$\text{Law of cosines: } a^2 = b^2 + c^2 - 2bc \cos \alpha$$

$$\text{Law of tangents: } \frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$$

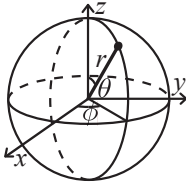
2.4.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

2.4.3 Spherical coordinates



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \text{atan2}(y, x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\frac{d}{dx} \arcsin x = \frac{1}{\sqrt{1 - x^2}} \quad \frac{d}{dx} \arccos x = -\frac{1}{\sqrt{1 - x^2}}$$

$$\frac{d}{dx} \tan x = 1 + \tan^2 x \quad \frac{d}{dx} \arctan x = \frac{1}{1 + x^2}$$

$$\int \tan ax = -\frac{\ln |\cos ax|}{a} \quad \int x \sin ax = \frac{\sin ax - ax \cos ax}{a^2}$$

$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2} \text{erf}(x) \quad \int x e^{ax} dx = \frac{e^{ax}}{a^2} (ax - 1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \dots + n = \frac{n(n + 1)}{2}$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n + 1)(n + 1)}{6}$$

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n + 1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$$

$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$$

$$\sqrt{1 + x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\text{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1 - p)^{n - k}$$

$$\mu = np, \sigma^2 = np(1 - p)$$

$\text{Bin}(n, p)$ is approximately $\text{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $Fs(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1 - p)^{k-1}, \, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \, \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $Po(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \, \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $U(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}, \, \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $Exp(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets **A** and **G**, such that all states in **A** are absorbing ($p_{ii} = 1$), and all states in **G** leads to an absorbing state in **A**. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type. **Time:** $\mathcal{O}(\log N)$

```
c4d #include <bits/extc++.h>
e82 using namespace __gnu_pbds;

f07 template<class T>
e93 using Tree = tree<T, null_type, less<T>, rb_tree_tag,
cd2     tree_order_statistics_node_update>;

af6 void example() {
d3b     Tree<int> t, t2; t.insert(8);
e2d     auto it = t.insert(10).first;
b1d     assert(it == t.lower_bound(9));
c1f     assert(t.order_of_key(10) == 1);
ba5     assert(t.order_of_key(11) == 2);
c0b     assert(*t.find_by_order(0) == 8);
4eb     t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
782 }
```

IndexedSet.h

Description: Similar to OrderStatisticTree, but uses a different library which may be more portable. change less to less_equal for multiset (this breaks the erase(value) function)

```
774 #include <ext/pb_ds/assoc_container.hpp>
32f using namespace __gnu_pbds;

9fd template<class T>
7e5 using IndexedSet = tree<T, null_type, less<T>, rb_tree_tag,
bbd     tree_order_statistics_node_update>;

d77092, 7 lines

c4d #include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
204 struct chash { // large odd number for C
4b5     const uint64_t C = 11(4e18 * acos(0)) | 71;
1e4     ll operator()(ll x) const { return __builtin_bswap64(x*C);
    }
9b4 };
d77 __gnu_pbds::gp_hash_table<ll,int, chash> h({},{},{},{},{1<16
    });
```

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit. **Time:** $\mathcal{O}(\log N)$

```
0f4bdb, 19 lines

5ae struct Tree {
fdb     typedef int T;
9ef     static constexpr T unit = INT_MIN;
6cd     T f(T a, T b) { return max(a, b); } // (any associative fn
    )
026     vector<T> s; int n;
2b8     Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
954     void update(int pos, T val) {
945         for (s[pos += n] = val; pos /= 2;)
2f2             s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
17a     }
b7e     T query(int b, int e) { // query [b, e)
1c8         T ra = unit, rb = unit;
1ce         for (b += n, e += n; b < e; b /= 2, e /= 2) {
84b             if (b % 2) ra = f(ra, s[b++]);
d46             if (e % 2) rb = f(s[--e], rb);
339         }
3f6         return f(ra, rb);
c90     }
0f4 };
```

LazySegmentTree.h

Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory. **Usage:** Node* tr = new Node(v, 0, sz(v)); **Time:** $\mathcal{O}(\log N)$.

```
"../various/BumpAllocator.h"
34ecf5, 50 lines

1a8 const int inf = 1e9;
369 struct Node {
f58     Node *l = 0, *r = 0;
d01     int lo, hi, mset = inf, madd = 0, val = -inf;
317     Node(int lo, int hi):lo(lo),hi(hi){} // Large interval of -
inf
47f     Node(vi& v, int lo, int hi) : lo(lo), hi(hi) {
5da         if (lo + 1 < hi) {
8b7             int mid = lo + (hi - lo)/2;
```

```
62d         l = new Node(v, lo, mid); r = new Node(v, mid, hi);
a14         val = max(l->val, r->val);
ab0     }
7d6     else val = v[lo];
f45     }
dda     int query(int L, int R) {
892         if (R <= lo || hi <= L) return -inf;
89a         if (L <= lo && hi <= R) return val;
1f6         push();
e81         return max(l->query(L, R), r->query(L, R));
18e     }
a49     void set(int L, int R, int x) {
492         if (R <= lo || hi <= L) return;
1dc         if (L <= lo && hi <= R) mset = val = x, madd = 0;
065         else {
0c8             push(), l->set(L, R, x), r->set(L, R, x);
fee             val = max(l->val, r->val);
33c         }
efb     }
fa9     void add(int L, int R, int x) {
f29         if (R <= lo || hi <= L) return;
f79         if (L <= lo && hi <= R) {
91d             if (mset != inf) mset += x;
7f6             else madd += x;
7c0             val += x;
318         }
102         else {
22b             push(), l->add(L, R, x), r->add(L, R, x);
c59             val = max(l->val, r->val);
18f         }
587     }
c5a     void push() {
53c         if (!l) {
ce5             int mid = lo + (hi - lo)/2;
103             l = new Node(lo, mid); r = new Node(mid, hi);
ae6         }
588         if (mset != inf)
fa9             l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
39c         else if (madd)
305             l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
804     }
34e    };
```

UnionFind.h
Description: Disjoint-set data structure.
Time: $\mathcal{O}(\alpha(N))$

7aa27c, 14 lines

```
ba2 struct UF {
71e     vi e;
b71     UF(int n) : e(n, -1) {}
bb8     bool sameSet(int a, int b) { return find(a) == find(b); }
3ab     int size(int x) { return -e[find(x)]; }
3f6     int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
    }
cb8     bool join(int a, int b) {
bfd         a = find(a), b = find(b);
214         if (a == b) return false;
12e         if (e[a] > e[b]) swap(a, b);
204         e[a] += e[b]; e[b] = a;
9c5         return true;
a61     }
7aa    };
```

UnionFindRollback.h
Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: int t = uf.time(); ...; uf.rollback(t);
Time: $\mathcal{O}(\log(N))$

```
47a struct RollbackUF {
32c     vi e; vector<pii> st;
66f     RollbackUF(int n) : e(n, -1) {}
dfd     int size(int x) { return -e[find(x)]; }
f73     int find(int x) { return e[x] < 0 ? x : find(e[x]); }
821     int time() { return sz(st); }
154     void rollback(int t) {
d4a         for (int i = time(); i --> t;)
96b             e[st[i].first] = st[i].second;
933         st.resize(t);
e7f     }
3f4     bool join(int a, int b) {
9dd         a = find(a), b = find(b);
081         if (a == b) return false;
404         if (e[a] > e[b]) swap(a, b);
3aa         st.push_back({a, e[a]});
5f7         st.push_back({b, e[b]});
fa6         e[a] += e[b]; e[b] = a;
21e         return true;
f07     }
de4    };
```

SubMatrix.h
Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).
Usage: SubMatrix<int> m(matrix);
m.sum(0, 0, 2, 2); // top left 4 elements
Time: $\mathcal{O}(N^2 + Q)$

c59ada, 13 lines

```
4fc template<class T>
92b struct SubMatrix {
b96     vector<vector<T>>> p;
286     SubMatrix(vector<vector<T>>& v) {
03e         int R = sz(v), C = sz(v[0]);
393         p.assign(R+1, vector<T>(C+1));
0b5         rep(r,0,R) rep(c,0,C)
064             p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c]
    ];
aca     }
543     T sum(int u, int l, int d, int r) {
202         return p[d][r] - p[d][l] - p[u][r] + p[u][l];
5e3     }
c59    };
```

Matrix.h
Description: Basic operations on square matrices.
Usage: Matrix<int, 3> A;
A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}}};
array<int, 3> vec = {1,2,3};
vec = (A^N) * vec;

4da5a2, 26 lines

```
a95 template<class T, int N> struct Matrix {
3ef     typedef Matrix M;
e08     array<array<T, N>, N> d{};
c6c     M operator*(const M& m) const {
1aa         M a;
5aa         rep(i,0,N) rep(j,0,N)
6e4             rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
c95         return a;
7a1     }
945     array<T, N> operator*(const array<T, N>& vec) const {
c4c         array<T, N> ret{};
bc7         rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
be4         return ret;
f64     }
339     M operator^(ll p) const {
fd8         assert(p >= 0);
f42         M a, b(*this);
2a2         rep(i,0,N) a.d[i][i] = 1;
```

```
08e     while (p) {
b36         if (p&1) a = a*b;
ae5         b = b*b;
9cc         p >>= 1;
05c     }
13f     return a;
a87     }
4da    };
```

LineContainer.h
Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).
Time: $\mathcal{O}(\log N)$

Sec1c7, 30 lines

```
72c struct Line {
14c     mutable ll k, m, p;
0c4     bool operator<(const Line& o) const { return k < o.k; }
0dc     bool operator<(ll x) const { return p < x; }
7e3    };

746 struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
a3f     static const ll inf = LLONG_MAX;
671     ll div(ll a, ll b) { // floored division
fa8         return a / b - ((a ^ b) < 0 && a % b); }
1a9     bool isect(iterator x, iterator y) {
333         if (y == end()) return x->p = inf, 0;
120         if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
d6d         else x->p = div(y->m - x->m, x->k - y->k);
846         return x->p >= y->p;
31f     }
4fa     void add(ll k, ll m) {
ebc         auto z = insert({k, m, 0}), y = z++, x = y;
e18         while (isect(y, z)) z = erase(z);
56f         if (x != begin() && isect(--x, y)) isect(x, y = erase(y)
    );
6dc         while ((y = x) != begin() && (--x)->p >= y->p)
3f5             isect(x, erase(y));
4e2     }
809     ll query(ll x) {
d8b         assert(!empty());
143         auto l = *lower_bound(x);
881         return l.k * x + l.m;
5a0     }
8ec    };
```

Treap.h
Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.
Time: $\mathcal{O}(\log N)$

1754b4, 53 lines

```
b72 struct Node {
09c     Node *l = 0, *r = 0;
609     int val, y, c = 1;
1e3     Node(int val) : val(val), y(rand()) {}
829     void recalc();
daa    };

6c5 int cnt(Node* n) { return n ? n->c : 0; }
371 void Node::recalc() { c = cnt(l) + cnt(r) + 1; }

6b5 template<class F> void each(Node* n, F f) {
19c     if (n) { each(n->l, f); f(n->val); each(n->r, f); }
c7b    }

0d5 pair<Node*, Node*> split(Node* n, int k) {
818     if (!n) return {};
38e     if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
```

```
ffd      auto [L,R] = split(n->l, k);
d0f      n->l = R;
a93      n->recalc();
2a2      return {L, n};
d87    } else {
f6c      auto [L,R] = split(n->r,k - cnt(n->l) - 1); // and just
" k"

b25      n->r = L;
08a      n->recalc();
2ef      return {n, R};
163    }
b24  }

27f Node* merge(Node* l, Node* r) {
34d      if (!l) return r;
917      if (!r) return l;
907      if (l->y > r->y) {
67d          l->r = merge(l->r, r);
719          return l->recalc(), l;
27e      } else {
f27          r->l = merge(l, r->l);
ffc          return r->recalc(), r;
d58      }
a1f  }
```

```
ba8 Node* ins(Node* t, Node* n, int pos) {
28b      auto [l,r] = split(t, pos);
6ed      return merge(merge(l, n), r);
473 }
```

```
// Example application: move the range [l, r) to index k
43d void move(Node*& t, int l, int r, int k) {
dcf      Node *a, *b, *c;
b65      tie(a,b) = split(t, l); tie(b,c) = split(b, r - 1);
d86      if (k <= l) t = merge(ins(a, b, k), c);
5ef      else t = merge(a, ins(c, b, k - r));
175 }
```

FenwickTree.h

Description: Computes partial sums $a[0] + a[1] + \dots + a[pos - 1]$, and updates single elements $a[i]$, taking the difference between the old and new value.
Time: Both operations are $\mathcal{O}(\log N)$.

```
066 struct FT {
429     vector<ll> s;
890     FT(int n) : s(n) {}
a5a     void update(int pos, ll dif) { // a[pos] += dif
711         for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
a0c     }
404     ll query(int pos) { // sum of values in [0, pos)
b49         ll res = 0;
110         for (; pos > 0; pos &= pos - 1) res += s[pos-1];
ee7         return res;
585     }
580     int lower_bound(ll sum) { // min pos st sum of [0, pos] >=
sum
// Returns n if no sum is >= sum, or -1 if empty sum is

2ba     if (sum <= 0) return -1;
782     int pos = 0;
d79     for (int pw = 1 << 25; pw; pw >>= 1) {
662         if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
1f4             pos += pw, sum -= s[pos-1];
cb4     }
9c3     return pos;
923 }
e62 };
```

FenwickTree2d.h

Description: Computes sums $a[i,j]$ for all $i < I, j < J$, and increases single elements $a[i,j]$. Requires that the elements to be updated are known in advance (call `fakeUpdate()` before `init()`).
Time: $\mathcal{O}(\log^2 N)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

```
"FenwickTree.h"157f07.22 lines

9a3 struct FT2 {
f22     vector<vi> ys; vector<FT> ft;
5e2     FT2(int limx) : ys(limx) {}
8ab     void fakeUpdate(int x, int y) {
e22         for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
fe9     }
f14     void init() {
164         for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
1e4     }
874     int ind(int x, int y) {
2db         return (int)(lower_bound(all(ys[x]), y) - ys[x].begin())
; }
bb0     void update(int x, int y, ll dif) {
b18         for (; x < sz(ys); x |= x + 1)
5a5             ft[x].update(ind(x, y), dif);
c48     }
484     ll query(int x, int y) {
542         ll sum = 0;
953         for (; x; x &= x - 1)
c32             sum += ft[x-1].query(ind(x-1, y));
eb0         return sum;
266     }
157 };
```

RMQ.h

Description: Range Minimum Queries on an array. Returns $\min(V[a], V[a + 1], \dots V[b - 1])$ in constant time.
Usage: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`
Time: $\mathcal{O}(|V| \log |V| + Q)$

```
4fc template<class T>
14c struct RMQ {
b47     vector<vector<T>> jmp;
275     RMQ(const vector<T>& V) : jmp(1, V) {
722         for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
742             jmp.emplace_back(sz(V) - pw * 2 + 1);
e3f             rep(j,0,sz(jmp[k]))
c4d                 jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
398         }
dff     }
be4     T query(int a, int b) {
54f         assert(a < b); // or return inf if a == b
c82         int dep = 31 - __builtin_clz(b - a);
734         return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
63f     }
510 };
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).
Time: $\mathcal{O}(N\sqrt{Q})$

```
add1a12ef4.49 lines

ddb void add(int ind, int end) { ... } // add a[ind] (end = 0 or
1)
755 void del(int ind, int end) { ... } // remove a[ind]
fe2 int calc() { ... } // compute current answer

342 vi mo(vector<pii> Q) {
671     int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
e77     vi s(sz(Q)), res = s;
```

```
570 #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
9a8     iota(all(s), 0);
fb8     sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]);
});
d3c     for (int qi : s) {
bba         pii q = Q[qi];
dd1         while (L > q.first) add(--L, 0);
368         while (R < q.second) add(R++, 1);
006         while (L < q.first) del(L++, 0);
400         while (R > q.second) del(--R, 1);
8c7         res[qi] = calc();
d49     }
e19     return res;
7b2 }
```

```
be6 vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root
=0){
ec4     int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
010     vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
933     add(0, 0), in[0] = 1;
af3     auto dfs = [&](int x, int p, int dep, auto& f) -> void {
833         par[x] = p;
55f         L[x] = N;
2c1         if (dep) I[x] = N++;
f4f         for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
602         if (!dep) I[x] = N++;
fe8         R[x] = N;
7ac     };
786     dfs(root, -1, 0, dfs);
5e8 #define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk &
1))
96d     iota(all(s), 0);
64e     sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]);
});
6a3     for (int qi : s) rep(end,0,2) {
372         int &a = pos[end], b = Q[qi][end], i = 0;
c8a #define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
98d             else { add(c, end); in[c] = 1; } a = c; } \
72a         while (!(L[b] <= L[a] && R[a] <= R[b]))
c95             I[i++] = b, b = par[b];
24c         while (a != b) step(par[a]);
7db         while (i--) step(I[i]);
c41         if (end) res[qi] = calc();
44b     }
be2     return res;
a12 }
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

```
c9b7b0.17 lines

213 struct Poly {
640     vector<double> a;
aea     double operator()(double x) const {
b40         double val = 0;
1b7         for (int i = sz(a); i--;) (val *= x) += a[i];
374         return val;
f7a     }
187     void diff() {
462         rep(i,1,sz(a)) a[i-1] = i*a[i];
1e1         a.pop_back();
d44     }
cd4     void divroot(double x0) {
323         double b = a.back(), c; a.back() = 0;
06b         for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b
=c;
```

```
071      a.pop_back();
43b    }
c9b  };
```

PolyRoots.h

Description: Finds the real roots to a polynomial.

Usage: `polyRoots({{2,-3,1}},-1e9,1e9) // solve  $x^2-3x+2 = 0$`

Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

```

Polynomial.h" b00bfe, 23 lines
64a vector<double> polyRoots(Poly p, double xmin, double xmax) {
a63     if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
343     vector<double> ret;
2ac     Poly der = p;
840     der.diff();
105     auto dr = polyRoots(der, xmin, xmax);
31d     dr.push_back(xmin-1);
324     dr.push_back(xmax+1);
560     sort(all(dr));
501     rep(i,0,sz(dr)-1) {
d04         double l = dr[i], h = dr[i+1];
274         bool sign = p(l) > 0;
ea5         if (sign ^ (p(h) > 0)) {
cc4             rep(it,0,60) { // while (h - l > 1e-8)
40b                 double m = (l + h) / 2, f = p(m);
145                 if ((f <= 0) ^ sign) l = m;
8da                 else h = m;
4f1             }
f59             ret.push_back((l + h) / 2);
1c9         }
d5f     }
a51     return ret;
b00 }

```

PolyInterpolate.h

Description: Given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k / (n-1) * \pi)$, $k = 0 \dots n-1$.

Time: $\mathcal{O}(n^2)$

```

ae0 typedef vector<double> vd;
28c vd interpolate(vd x, vd y, int n) {
a3c     vd res(n), temp(n);
01c     rep(k,0,n-1) rep(i,k+1,n)
159         y[i] = (y[i] - y[k]) / (x[i] - x[k]);
ca9     double last = 0; temp[0] = 1;
58f     rep(k,0,n) rep(i,0,n) {
9c9         res[i] += y[k] * temp[i];
e58         swap(last, temp[i]);
eb1         temp[i] -= last * x[k];
8c4     }
d40     return res;
08b }
```

BerlekampMassey.h

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.

Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`

Time: $\mathcal{O}(N^2)$

```

c10 vector<ll> berlekampMassey(vector<ll> s) {
4a8     int n = sz(s), L = 0, m = 0;
102     vector<ll> C(n), B(n), T;
b21     C[0] = B[0] = 1;

b79     ll b = 1;
241     rep(i,0,n) { ++m;

```

```

e84      ll d = s[i] % mod;
7e7      rep(j,l,L+1) d = (d + C[j] * s[i - j]) % mod;
f1e      if (!d) continue;
b3b      T = C; ll coef = d * modpow(b, mod-2) % mod;
b57      rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
a5a      if (2 * L > i) continue;
247      L = i + 1 - L; B = T; b = d; m = 0;
3dc  }

```

```

1dea C.resize(L + 1); C.erase(C.begin());
5fe  for (ll& x : C) x = (mod - x) % mod;
3f3  return C;
965 }

```

LinearRecurrence.h

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i - j - 1]tr[j]$, given $S[0 \dots \geq n - 1]$ and $tr[0 \dots n - 1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.

Usage: `linearRec({0, 1}, {1, 1}, k)` // k'th Fibonacci number

Time: $\mathcal{O}(n^2 \log k)$

```

16c typedef vector<ll> Poly;
a29 ll linearRec(Poly S, Poly tr, ll k) {
465     int n = sz(tr);

bb1     auto combine = [&](Poly a, Poly b) {
d3c         Poly res(n * 2 + 1);
935         rep(i,0,n+1) rep(j,0,n+1)
831             res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
334         for (int i = 2 * n; i > n; --i) rep(j,0,n)
b5c             res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) %
mod;
697         res.resize(n + 1);
6c0         return res;
da8     };

5d8     Poly pol(n + 1, e(pol));
0b6     pol[0] = e[1] = 1;

574     for (++k; k; k /= 2) {
3da         if (k % 2) pol = combine(pol, e);
bb2         e = combine(e, e);
b65     }

52d     ll res = 0;
a58     rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
8b5     return res;
f4e }

```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is eps . Works equally well for maximization with a small change in the code. See Ternary-Search.h in the Various chapter for a discrete version.

```
Usage: double func(double x) { return 4+x+.3*x*x; }
```

```
double xmin = gss(-1000,1000,func);
```

Time: $\mathcal{O}(\log((b-a)/\epsilon))$

```

ebl  double gss(double a, double b, double (*f)(double)) {
c68  double r = (sqrt(5)-1)/2, eps = 1e-7;
2a1  double x1 = b - r*(b-a), x2 = a + r*(b-a);
r89  double f1 = f(x1), f2 = f(x2);
40b  while (b-a > eps) {
071    if (f1 < f2) { //change to > to find maximum
012      b = x2; x2 = x1; f2 = f1;
4ed      x1 = b - r*(b-a); f1 = f(x1);
c73    } else {

```

```

62b      a = x1; x1 = x2; f1 = f2;
0fa      x2 = a + r*(b-a); f2 = f(x2);
821    }
39c  return a;
31d }

```

HillClimbing.h

Description: Poor man's optimization for unimodal functions, 8eeef, 14 lines

```
9eb typedef array<double, 2> P;
```

```

170 template<class F> pair<double, P> hillClimb(P start, F f) {
18b   pair<double, P> cur(f(start), start);
68a   for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
1a2     rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
d5b       P p = cur.second;
aaa       p[0] += dx*jmp;
bd4       p[1] += dy*jmp;
64a       cur = min(cur, make_pair(f(p), p));
932     }
523   }
34f   return cur;
8ee }

```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4 , although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

47361c, 7 lines

```

044 template<class F>
751 double quad(double a, double b, F f, const int n = 1000) {
840     double h = (b - a) / 2 / n, v = f(a) + f(b);
b84     rep(i,1,n*2)
e93         v += f(a + i*h) * (i%1 ? 4 : 2);
df3     return v * h / 3;
475 }

```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.

```
Usage: double sphereVolume = quad(-1, 1, [])(double x) {
return quad(-1, 1, [&](double y) {
return quad(-1, 1, [&](double z) {
return x*x + y*y + z*z < 1; }));});});
92dd79 15 lines
```

```
070 typedef double d;  
459 #define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6
```

```

e42 template <class F>
e70 d rec(F& f, d a, d b, d eps, d S) {
e8a     d c = (a + b) / 2;
bdc     d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
b97     if (abs(T - S) <= 15 * eps || b - a < 1e-10)
3f5         return T + (T - S) / 15;
4d1     return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2,
S2);
a81 }
e8c template<class F>
248 d quad(d a, d b, F f, d eps = 1e-8) {
868     return rec(f, a, b, eps, S(a, b));
92d }

```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.

Usage: vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
Time: $\mathcal{O}(NM * \# \text{pivots})$, where a pivot may be e.g. an edge relaxation.
 $\mathcal{O}(2^n)$ in the general case.

```
aa8530, 68 lines
943 typedef double T; // long double, Rational, double + modKp
    >...
4a7 typedef vector<T> vd;
194 typedef vector<vd> vvd;

629 const T eps = 1e-8, inf = 1/.0;
20r #define MP make_pair
80a #define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s]))
    s=j

004 struct LPSolver {
34f     int m, n;
a8b     vi N, B;
a50     vvd D;

e8e     LPSolver(const vvd& A, const vd& b, const vd& c) :
09e         m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
a00             rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
eab             rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i]
];}

03b             rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
4c2             N[n] = -1; D[m+1][n] = 1;
dca             }

d2d     void pivot(int r, int s) {
72c         T *a = D[r].data(), inv = 1 / a[s];
93b         rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
a86             T *b = D[i].data(), inv2 = b[s] * inv;
c1f             rep(j,0,n+2) b[j] -= a[j] * inv2;
ee2             b[s] = a[s] * inv2;
d7             }
d3c         rep(j,0,n+2) if (j != s) D[r][j] *= inv;
9e2         rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
6bf         D[r][s] = inv;
b34         swap(B[r], N[s]);
193     }

ede     bool simplex(int phase) {
f69         int x = m + phase - 1;
0aa         for (;;) {
8b6             int s = -1;
96f             rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
e72             if (D[x][s] >= -eps) return true;
fed             int r = -1;
a7d             rep(i,0,m) {
f65                 if (D[i][s] <= eps) continue;
01f                 if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
8af                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
170             }
23b             if (r == -1) return false;
100             pivot(r, s);
d81         }
62b     }
```

```
48a     T solve(vd &x) {
b07         int r = 0;
cc8         rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
dc3         if (D[r][n+1] < -eps) {
fbf             pivot(r, n);
09c             if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
6b2             rep(i,0,m) if (B[i] == -1) {
9aa                 int s = 0;
db9                 rep(j,1,n+1) ltj(D[i]);
```

```
d11         pivot(i, s);
213     }
36d }
e28 bool ok = simplex(1); x = vd(n);
002 rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
8dd return ok ? D[m][n+1] : inf;
bc3 }
aa8 ;;
```

4.3 Matrices

Determinant.h
Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

```
bd5cec, 15 lines
e36 double det(vector<vector<double>>& a) {
590     int n = sz(a); double res = 1;
d90     rep(i,0,n) {
4bd         int b = i;
309         rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
c6c         if (i != b) swap(a[i], a[b]), res *= -1;
658         res *= a[i][i];
390         if (res == 0) return 0;
15f         rep(j,i+1,n) {
356             double v = a[j][i] / a[i][i];
979             if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
ebf         }
aa3     }
7fe     return res;
bd5 }
```

IntDeterminant.h
Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}(N^3)$

```
3313dc, 18 lines
031 const ll mod = 12345;
ea0 ll det(vector<vector<ll>>& a) {
aea     int n = sz(a); ll ans = 1;
c9d     rep(i,0,n) {
cab         rep(j,i+1,n) {
4f6             while (a[j][i] != 0) { // gcd step
155                 ll t = a[i][i] / a[j][i];
f94                 if (t) rep(k,i,n)
618                     a[i][k] = (a[i][k] - a[j][k] * t) % mod;
4d6                 swap(a[i], a[j]);
cbb                 ans *= -1;
3e9             }
7ef         }
717         ans = ans * a[i][i] % mod;
c4c         if (!ans) return 0;
666     }
cd2     return (ans + mod) % mod;
331 }
```

SolveLinear.h
Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2m)$

```
44c9ab, 38 lines
ae0 typedef vector<double> vd;
178 const double eps = 1e-12;

dbd int solveLinear(vector<vd>& A, vd& b, vd& x) {
2cf     int n = sz(A), m = sz(x), rank = 0, br, bc;
61a     if (n) assert(sz(A[0]) == m);
274     vi col(m); iota(all(col), 0);

27c     rep(i,0,n) {
cdb         double v, bv = 0;
```

```
9bb         rep(r,i,n) rep(c,i,m)
889             if ((v = fabs(A[r][c])) > bv)
4ca                 br = r, bc = c, bv = v;
236         if (bv <= eps) {
008             rep(j,i,n) if (fabs(b[j]) > eps) return -1;
b9e             break;
e8d         }
e25         swap(A[i], A[br]);
f84         swap(b[i], b[br]);
b1e         swap(col[i], col[bc]);
0be         rep(j,0,n) swap(A[j][i], A[j][bc]);
bc2         bv = 1/A[i][i];
292         rep(j,i+1,n) {
416             double fac = A[j][i] * bv;
f8d             b[j] -= fac * b[i];
fe2             rep(k,i+1,m) A[j][k] -= fac*A[i][k];
34d         }
cc5         rank++;
66c     }
```

```
5f0     x.assign(m, 0);
21a     for (int i = rank; i--;) {
5fa         b[i] /= A[i][i];
9d7         x[col[i]] = b[i];
a0b         rep(j,0,i) b[j] -= A[j][i] * b[i];
55e     }
ec3     return rank; // (multiple solutions if rank < m)
44c }
```

SolveLinear2.h
Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

```
08e495, 7 lines
" SolveLinear.h"
f94 rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
3b9 x.assign(m, undefined);
45b rep(i,0,rank) {
22b     rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
468     x[col[i]] = b[i] / A[i][i];
08e fail;; }
```

SolveLinearBinary.h
Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .
Time: $\mathcal{O}(n^2m)$

```
fa2d7a, 34 lines
983 typedef bitset<1000> bs;

1dc int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
cda     int n = sz(A), rank = 0, br;
d90     assert(m <= sz(x));
b3f     vi col(m); iota(all(col), 0);
ede     rep(i,0,n) {
1de         for (br=i; br<n; ++br) if (A[br].any()) break;
af7         if (br == n) {
f71             rep(j,i,n) if(b[j]) return -1;
4a2             break;
84b         }
bb0         int bc = (int)A[br]._Find_next(i-1);
95e         swap(A[i], A[br]);
947         swap(b[i], b[br]);
df3         swap(col[i], col[bc]);
31f         rep(j,0,n) if (A[j][i] != A[j][bc]) {
8c1             A[j].flip(i); A[j].flip(bc);
bf5         }
e5b         rep(j,i+1,n) if (A[j][i]) {
dca             b[j] ^= b[i];
7a6             A[j] ^= A[i];
083         }
```



```
c27     rank++;
4de   }

d49   x = bs();
3e6   for (int i = rank; i--;) {
6b7       if (!b[i]) continue;
c22       x[col[i]] = 1;
17b       rep(j,0,i) b[j] ^= A[j][i];
fe1   }
df4   return rank; // (multiple solutions if rank < m)
fa2 }
```

MatrixInverse.h

Description: Invert matrix A . Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \bmod p$, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

ebfff6, 35 lines

```
4b5 int matInv(vector<vector<double>>& A) {
e91     int n = sz(A); vi col(n);
2e6     vector<vector<double>> tmp(n, vector<double>(n));
9a9     rep(i,0,n) tmp[i][i] = 1, col[i] = i;

8ec     rep(i,0,n) {
a71         int r = i, c = i;
3ff         rep(j,i,n) rep(k,i,n)
c8b             if (fabs(A[j][k]) > fabs(A[r][c]))
6b4                 r = j, c = k;
baa         if (fabs(A[r][c]) < 1e-12) return i;
748         A[i].swap(A[r]); tmp[i].swap(tmp[r]);
c48         rep(j,0,n)
6e2             swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
6ce         swap(col[i], col[c]);
59c         double v = A[i][i];
e17         rep(j,i+1,n) {
1c2             double f = A[j][i] / v;
3cc             A[j][i] = 0;
9da             rep(k,i+1,n) A[j][k] -= f*A[i][k];
293             rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
4b5         }
f7a         rep(j,i+1,n) A[i][j] /= v;
678         rep(j,0,n) tmp[i][j] /= v;
bbe         A[i][i] = 1;
cd3     }

28e     for (int i = n-1; i > 0; --i) rep(j,0,i) {
973         double v = A[j][i];
b37         rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
fd4     }

097     rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
898     return n;
ebf }
```

MatrixInverse-mod.h

Description: Invert matrix A modulo a prime. Returns rank; result is stored in A unless singular (rank < n). For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \bmod p$, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

"../number-theory/ModPow.h"

0b7b13, 37 lines

```
702 int matInv(vector<vector<ll>>& A) {
8d1     int n = sz(A); vi col(n);
ff2     vector<vector<ll>> tmp(n, vector<ll>(n));
ebd     rep(i,0,n) tmp[i][i] = 1, col[i] = i;

4c7     rep(i,0,n) {
```

```
196     int r = i, c = i;
163     rep(j,i,n) rep(k,i,n) if (A[j][k]) {
843         r = j; c = k; goto found;
670     }
43b     return i;
793 found:
6f7     A[i].swap(A[r]); tmp[i].swap(tmp[r]);
013     rep(j,0,n)
994         swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
f48     swap(col[i], col[c]);
a33     ll v = modpow(A[i][i], mod - 2);
221     rep(j,i+1,n) {
4dc         ll f = A[j][i] * v % mod;
820         A[j][i] = 0;
191         rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
203         rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) % mod
;
3af     }
402     rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
6e1     rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
709     A[i][i] = 1;
b5f }

9c0     for (int i = n-1; i > 0; --i) rep(j,0,i) {
8a3         ll v = A[j][i];
fb9         rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
597     }

765     rep(i,0,n) rep(j,0,n)
244         A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0)*
mod;
191     return n;
0b7 }
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither tr nor the check for diag[i] == 0 is needed.

Time: $\mathcal{O}(N)$

8f9fa8, 26 lines

```
943 typedef double T;
b20 vector<T> tridiagonal(vector<T> diag, const vector<T>& super
,
f81     const vector<T>& sub, vector<T> b) {
52e     int n = sz(b); vi tr(n);
399     rep(i,0,n-1) {
a25         if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] ==
0
564             b[i+1] -= b[i] * diag[i+1] / super[i];
018             if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
560             diag[i+1] = sub[i]; tr[++i] = 1;
ad9         } else {
```

```
e9d     diag[i+1] -= super[i]*sub[i]/diag[i];
133     b[i+1] -= b[i]*sub[i]/diag[i];
25f     }
7da }
db7     for (int i = n; i--;) {
ff8         if (tr[i]) {
148             swap(b[i], b[i-1]);
c73             diag[i-1] = diag[i];
6bd             b[i] /= super[i-1];
9a7         } else {
2fb             b[i] /= diag[i];
a82             if (i) b[i-1] -= b[i]*super[i-1];
94e         }
4f7     }
b1f     return b;
8f9 }
```

4.4 Fourier transforms

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_a a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

00ced6, 35 lines

```
bcc typedef complex<double> C;
b05 typedef vector<double> vd;
760 void fft(vector<C>& a) {
547     int n = sz(a), L = 31 - __builtin_clz(n);
1ec     static vector<complex<long double>> R(2, 1);
1e9     static vector<C> rt(2, 1); // (^ 10% faster if double)
beb     for (static int k = 2; k < n; k *= 2) {
af1         R.resize(n); rt.resize(n);
69a         auto x = polar(1.0L, acos(-1.0L) / k);
148         rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
42e     }
d8b     vi rev(n);
394     rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
8af     rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
14a     for (int k = 1; k < n; k *= 2)
9f2         for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
6e7             C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-
rolled)
e0d             a[i + j + k] = a[i + j] - z;
63e             a[i + j] += z;
865         }
3b9 }
ab9 vd conv(const vd& a, const vd& b) {
7ee     if (a.empty() || b.empty()) return {};
cc2     vd res(sz(a) + sz(b) - 1);
ef3     int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
d6c     vector<C> in(n), out(n);
d34     copy(all(a), begin(in));
ea3     rep(i,0,sz(b)) in[i].imag(b[i]);
c8e     fft(in);
e63     for (C& x : in) x *= x;
fb2     rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
e2d     fft(out);
989     rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
922     return res;
00c }
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = A + B $ (twice as slow as NTT or FFT)	
"FastFourierTransform.h"	b82773, 22 lines
<pre>192 typedef vector<ll> vl; 1db template<int M> vl convMod(const vl &a, const vl &b) { ffe if (a.empty() b.empty()) return {}; 909 vl res(sz(a) + sz(b) - 1); 2c4 int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M)); 21d vector<C> L(n), R(n), outs(n), outl(n); ff2 rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut); f13 rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut); f8a fft(L), fft(R); 747 rep(i,0,n) { 153 int j = -i & (n - 1); a18 outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n); 1a9 outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / li; 455 } 67d fft(outl), fft(outs); 086 rep(i,0,sz(res)) { 8bd ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5); 9ac ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5); 0af res[i] = ((av % M * cut + bv) % M * cut + cv) % M; 26b } 94c return res; b82 }</pre>	

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^ab + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFTMod. conv(a, b) = c, where $c[x] = \sum a[i]b[x - i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$

"...number-theory/ModPow.h"	ced03d, 35 lines
<pre>b5e const ll mod = (119 << 23) + 1, root = 62; // = 998244353 // For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21 // and 483 << 21 (same root). The last two are > 10^9. 745 typedef vector<ll> vl; 0ca void ntt(vl &a) { c96 int n = sz(a), L = 31 - __builtin_clz(n); 7bd static vl rt(2, 1); 668 for (static int k = 2, s = 2; k < n; k *= 2, s++) { 4c5 rt.resize(n); 175 ll z[] = {1, modpow(root, mod >> s)}; 292 rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod; 5fa } 3ee vi rev(n); 78d rep(i,0,n) rev[i] = (rev[i / 2] (i & 1) << L) / 2; 158 rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]); 225 for (int k = 1; k < n; k *= 2) 61b for (int i = 0; i < n; i += 2 * k) rep(j,0,k) { 64c ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j]; cba a[i + j + k] = ai - z + (z > ai ? mod : 0); 4b5 ai += (ai + z >= mod ? z - mod : z); 35d } 29a } bba vl conv(const vl &a, const vl &b) { 400 if (a.empty() b.empty()) return {}; ac0 int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s), cb0 n = 1 << B; 10d int inv = modpow(n, mod - 2); 5e3 vl L(a), R(b), out(n); 8e3 L.resize(n), R.resize(n); 641 ntt(L), ntt(R); c16 rep(i,0,n) 1c4 out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod; 4af ntt(out);</pre>	

70c	return {out.begin(), out.begin() + s};
ced }	

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

	464cf3, 16 lines
<pre>ac2 void FST(vi& a, bool inv) { 99f for (int n = sz(a), step = 1; step < n; step *= 2) { fb2 for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) { 982 int &u = a[j], &v = a[j + step]; tie(u, v) = ae8 inv ? pii(v - u, u) : pii(v, u + v); // AND 6fa inv ? pii(v, u - v) : pii(u + v, u); // OR 2e1 pii(u + v, u - v); // XOR 398 } 343 } 808 if (inv) for (int& x : a) x /= sz(a); // XOR only 57e } c24 vi conv(vi a, vi b) { 108 FST(a, 0); FST(b, 0); 056 rep(i,0,sz(a)) a[i] *= b[i]; e2a FST(a, 1); return a; 464 }</pre>	

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

"euclid.h"	ealac7, 19 lines
<pre>4eb const ll mod = 17; // change to something else 665 struct Mod { e58 ll x; 42d Mod(ll y) : Mod(y%mod+mod,0){} 789 Mod(ll y,int) : x(y<mod?y:y-mod){} 976 Mod operator+(Mod b) { return {x + b.x,0}; } ef5 Mod operator-(Mod b) { return {x - b.x + mod,0}; } 639 Mod operator*(Mod b) { return {x * b.x % mod,0}; } f1f Mod operator/(Mod b) { return *this * invert(b); } b85 Mod invert(Mod a) { 8f4 ll x, y, g = euclid(a.x, mod, x, y); c47 assert(g == 1); return x; 193 } 31b Mod operator^(ll e) { 516 if (!e) return 1; 019 Mod r = *this ^ (e / 2); r = r * r; ce3 return e&1 ? *this * r : r; cbc } eal };</pre>	

ModInverse.h

Description: Pre-computation of modular inverses. Assumes $\text{LIM} \leq \text{mod}$ and that mod is a prime.

	6f684f, 3 lines
<pre>88a const ll mod = 1000000007, LIM = 200000; 3f0 ll* inv = new ll[LIM] - 1; inv[1] = 1; 6f6 rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;</pre>	

ModPow.h

	b83e45, 8 lines
<pre>e2e const ll mod = 1000000007; // faster if const cce ll modpow(ll b, ll e) {</pre>	

cd3	ll ans = 1;
8bc	for (; e; b = b * b % mod, e /= 2)
c96	if (e & 1) ans = ans * b % mod;
a23	return ans;
b83 }	

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$

	c040b8, 11 lines
<pre>0c8 ll modLog(ll a, ll b, ll m) { b25 ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1; bb2 unordered_map<ll, ll> A; 390 while (j <= n && (e = f = e * a % m) != b % m) 260 A[e * b % m] = j++; 2d9 if (e == b % m) return j; 36a if (__gcd(m, e) == __gcd(m, b)) 457 rep(i,2,n+2) if (A.count(e = e * f % m)) 4d3 return n * i - A[e]; a9a return -1; c04 }</pre>	

ModSum.h

Description: Sums of mod'ed arithmetic progressions.

$\text{modsum}(\text{to}, c, k, m) = \sum_{i=0}^{\text{to}-1} (ki + c) \% m$. divsum is similar but for floored division.

Time: $\log(m)$, with a large constant.

	5c5bc5, 16 lines
<pre>f4c typedef unsigned long long ull; 6bd ull sumsq(ull to) { return to / 2 * ((to-1) 1); }</pre>	

c2a	ull divsum(ull to, ull c, ull k, ull m) {
df3	ull res = k / m * sumsq(to) + c / m * to;
45f	k %= m; c %= m;
d4b	if (!k) return res;
da4	ull to2 = (to * k + c) / m;
c69	return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
4a5 }	
8eb	ll modsum(ull to, ll c, ll k, ll m) {
290	c = ((c % m) + m) % m;
148	k = ((k % m) + m) % m;
f53	return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
5c5 }	

ModMulLL.h

Description: Calculate $a \cdot b \pmod c$ (or $a^b \pmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

	bbbd8f, 11 lines
<pre>f4c typedef unsigned long long ull; 92e ull modmul(ull a, ull b, ull M) { 00a ll ret = a * b - M * ull(1.L / M * a * b); 21b return ret + M * (ret < 0) - M * (ret >= (ll)M); a9c } 438 ull modpow(ull b, ull e, ull mod) { c04 ull ans = 1; aea for (; e; b = modmul(b, b, mod), e /= 2) f5a if (e & 1) ans = modmul(ans, b, mod); 6d3 return ans; bbb }</pre>	

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModPow.h"	19a793, 24 lines
<pre>a77 ll sqrt(ll a, ll p) { e14 a %= p; if (a < 0) a += p;</pre>	

```
c0a  if (a == 0) return 0;
c5c  assert(modpow(a, (p-1)/2, p) == 1); // else no solution
473  if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
6aa  ll s = p - 1, n = 2;
b4a  int r = 0, m;
9e3  while (s % 2 == 0)
968      ++r, s /= 2;
2d4  while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
94d  ll x = modpow(a, (s + 1) / 2, p);
063  ll b = modpow(a, s, p), g = modpow(n, s, p);
c5e  for (;;) r = m) {
dea      ll t = b;
356      for (m = 0; m < r && t != 1; ++m)
2d5          t = t * t % p;
ab4      if (m == 0) return x;
8ca      ll gs = modpow(g, 1LL << (r - m - 1), p);
4c1      g = gs * gs % p;
2f0      x = x * gs % p;
198      b = b * g % p;
ac3  }
19a }
```

5.2 Primality

Eratosthenes.h

Description: Prime sieve for generating all primes up to a certain limit. isprime[i] is true iff i is a prime.
Time: lim=100'000'000 ≈ 0.8 s. Runs 30% faster if only odd indices are stored.

7c144c, 11 lines

```
129 const int MAX_PR = 5'000'000;
4c8 bitset<MAX_PR> isprime;
e30 vi eratosthenesSieve(int lim) {
b80     isprime.set(); isprime[0] = isprime[1] = 0;
b71     for (int i = 4; i < lim; i += 2) isprime[i] = 0;
6c6     for (int i = 3; i*i < lim; i += 2) if (isprime[i])
4c1         for (int j = i*i; j < lim; j += i*2) isprime[j] = 0;
081     vi pr;
98b     rep(i,2,lim) if (isprime[i]) pr.push_back(i);
379     return pr;
7c1 }
```

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 ≈ 1.5s

6b2912, 20 lines

```
2d0 const int LIM = 1e6;
04d bitset<LIM> isPrime;
7fd vi eratosthenes() {
a11     const int S = (int)round(sqrt(LIM)), R = LIM / 2;
058     vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1))
;
819     vector<pii> cp;
d3b     for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
97f         cp.push_back({i, i * i / 2});
579         for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
e31     }
91c     for (int L = 1; L <= R; L += S) {
883         array<bool, S> block{};
7bc         for (auto &[p, idx] : cp)
1df             for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
ac0         rep(i,0,min(S, R - L))
3db             if (!block[i]) pr.push_back((L + i) * 2 + 1);
4de     }
d77     for (int i : pr) isPrime[i] = 1;
710     return pr;
6b2 }
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \bmod c$.

"ModMulLL.h"60dcd1, 12 lines

```
da4 bool isPrime(ull n) {
6e0     if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
ad4     ull A[] = {2, 325, 9375, 28178, 450775, 9780504,
1795265022},
13b         s = __builtin_ctzll(n-1), d = n >> s;
60a     for (ull a : A) { // ^ count trailing zeroes
29e         ull p = modpow(a%n, d, n), i = s;
4ab         while (p != 1 && p != n - 1 && a % n && i--)
f79             p = modmul(p, p, n);
56f         if (p != n-1 && i != s) return 0;
1fa     }
3c0     return 1;
60d }
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h"d8d98d, 18 lines

```
7eb ull pollard(ull n) {
567     ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
12d     auto f = [&](ull x) { return modmul(x, x, n) + i; };
6b0     while (t++ % 40 || __gcd(prd, n) == 1) {
47d         if (x == y) x = ++i, y = f(x);
a82         if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
5dc         x = f(x), y = f(f(y));
0b4     }
8bf     return __gcd(prd, n);
cd2 }
c37 vector<ull> factor(ull n) {
630     if (n == 1) return {};
74d     if (isPrime(n)) return {n};
09e     ull x = pollard(n);
0d9     auto l = factor(x), r = factor(n / x);
a52     l.insert(l.end(), all(r));
26c     return l;
d8d }
```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `_gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

33ba8f, 5 lines

```
c22 ll euclid(ll a, ll b, ll &x, ll &y) {
fda     if (!b) return x = 1, y = 0, a;
d3d     ll d = euclid(b, a % b, y, x);
05a     return y -= a/b * x, d;
33b }
```

CRT.h

Description: Chinese Remainder Theorem.

crt(a, m, b, n) computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

"euclid.h"04d93a, 7 lines

```
24a ll crt(ll a, ll m, ll b, ll n) {
6cb     if (n > m) swap(a, b), swap(m, n);
8f5     ll x, y, g = euclid(m, n, x, y);
742     assert((a - b) % g == 0); // else no solution
eae     x = (b - a) % n * x % n / g * m + a;
```

```
000     return x < 0 ? x + m*n/g : x;
04d }
```

5.3.1 Bézout's identity

For $a \neq, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: Euler's ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p$ prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$. $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k, n) = 1} k = n\phi(n)/2, n > 1$.
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

cf7d6d, 8 lines

```
fd7 const int LIM = 5000000;
b4b int phi[LIM];
```

```
e30 void calculatePhi() {
70b     rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
9fb     for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
467         for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
cf7 }
```

PrimitiveRoot.h

Description: Finds the smallest Primitive Root of a given prime number. Guaranteed to work for numbers up to 10^{18} .

Time: $\mathcal{O}\left(p^{1/4}\right)$ expected time

"Factor.h"a6410d, 22 lines

```
80e ll prim_root(ll p) {
9d1     if (p == 2) return 1;
576     auto facts = factor(p - 1);

b5e     sort(facts.begin(), facts.end());
f23     facts.erase(unique(facts.begin(), facts.end()), facts.
end());

    // if p is not prime (4, p^k, 2*p^k)
    // update phi and grant gcd(res, n) = 1
    ll phi = p - 1;
    for (ll res = 2; res < p; ++res) {
        bool ok = true;
        for (ll f : facts) {
            if (modpow(res, phi / f, p) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return res;
    }
    return -1;
a64 }
```

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a 's eventually become cyclic.
Time: $\mathcal{O}(\log N)$

```
dd6c5e, 21 lines
070 typedef double d; // for N ~ 1e7; long double for N ~ 1e9
d72 pair<ll, ll> approximate(d x, ll N) {
709     ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
63f     for (;;) {
32b         ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf)
,
82c         a = (ll)floor(y), b = min(a, lim),
12b         NP = b*P + LP, NQ = b*Q + LQ;
426     if (a > b) {
        // If b > a/2, we have a semi-convergent that gives
        us a
        // better approximation; if b = a/2, we *may* have
        one.
        // Return {P, Q} here for a more canonical
        approximation.
f5e     return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q))
?
3c2         make_pair(NP, NQ) : make_pair(P, Q);
451     }
e56     if (abs(y = 1/(y - (d)a)) > 3*N) {
329         return {NP, NQ};
ec2     }
db8     LP = P; P = NP;
ed0     LQ = Q; Q = NQ;
a15 }
dd6 }
```

FracBinarySearch.h

Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}
Time: $\mathcal{O}(\log(N))$

```
27ab3e, 25 lines
fea struct Frac { ll p, q; };

339 template<class F>
94c Frac fracBS(F f, ll N) {
386     bool dir = 1, A = 1, B = 1;
832     Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
7f7     if (f(lo)) return lo;
eb7     assert(f(hi));
662     while (A || B) {
f6e         ll adv = 0, step = 1; // move hi if dir, else lo
ea2         for (int si = 0; step; (step *= 2) >= si) {
306             adv += step;
306             Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
aee             if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
8a8                 adv -= step; si = 2;
a40             }
d35         }
ce7         hi.p += lo.p * adv;
eed         hi.q += lo.q * adv;
758         dir = !dir;
828         swap(lo, hi);
fc8         A = B; B = !adv;
2c9     }
b6f     return dir ? hi : lo;
27a }
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$$\sum_{d \mid n} d = O(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

MobiusFunction.h

Description: Finds the mobius function
Time: $\mathcal{O}(n \log(\log(n)))$

```
366664, 11 lines
4e1 char meb[MAX];

ced void calculateMobius(int lim) {
020     for (int i = 2; i <= lim; i++) meb[i] = 2;
9de     meb[1] = 1;
87c     for (int i = 2; i <= lim; i++) if (meb[i] == 2)
886         for (int j = i; j <= lim; j += i) if (meb[j]) {
580             if (meb[j] == 2) meb[j] = 1;
d57             meb[j] *= j/i%i ? -1 : 0;
6ff         }
366 }
```

Mobius Inversion:

$$g(n) = \sum_{d \mid n} f(d) \Leftrightarrow f(n) = \sum_{d \mid n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$$\sum_{d \mid n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n \mid d} f(d) \Leftrightarrow f(n) = \sum_{n \mid d} \mu(d/n) g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.)
Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

```
044568, 6 lines
aeb int permToInt(vi& v) {
d3f     int use = 0, i = 0, r = 0;
e40     for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x
)),
b90         use |= 1 << x; // (note: minus, not
~!)
cf9     return r;
044 }
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp \left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k \mid n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0)=1, \; p(n)=\sum_{k\in\mathbb{Z}\setminus\{0\}}(-1)^{k+1}p(n-k(3k-1)/2)$$

$$p(n)\sim 0.145/n\cdot\exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\sim 2\text{e}5$	$\sim 2\text{e}8$

6.2.2 Lucas’ Theorem

Let n,m be non-negative integers and p a prime. Write $n=n_kp^k+...+n_1p+n_0$ and $m=m_kp^k+...+m_1p+m_0$. Then $\binom{n}{m}\equiv\prod_{i=0}^k\binom{n_i}{m_i}\pmod{p}$.

6.2.3 Binomials

multinomial.h	
Description: Computes $\binom{k_1+\cdots+k_n}{k_1,k_2,\ldots,k_n}=\frac{(\sum k_i)!}{k_1!k_2!\ldots k_n!}$.	a0a312, 5 lines
<pre>f7d ll multinomial(vi& v) { bf3 ll c = 1, m = v.empty() ? 1 : v[0]; 0eb rep(i,1,sz(v)) rep(j,0,v[i]) c = c * ++m / (j+1); efe return c; a0a }</pre>	

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t)=\frac{t}{e^t-1}$ (FFT-able).
 $B[0,\ldots]=[1,-\frac{1}{2},\frac{1}{6},0,-\frac{1}{30},0,\frac{1}{42},\ldots]$

Sums of powers:

$$\sum_{i=1}^n n^m=\frac{1}{m+1}\sum_{k=0}^m\binom{m+1}{k}B_k\cdot(n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^\infty f(i)=\int_m^\infty f(x)dx-\sum_{k=1}^\infty\frac{B_k}{k!}f^{(k-1)}(m)$$
$$\approx\int_m^\infty f(x)dx+\frac{f(m)}{2}-\frac{f'(m)}{12}+\frac{f'''(m)}{720}+O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k), \; c(0,0)=1$$
$$\sum_{k=0}^nc(n,k)x^k=x(x+1)\ldots(x+n-1)$$

$$c(8,k)=8,0,5040,13068,13132,6769,1960,322,28,1$$
$$c(n,2)=0,0,1,3,11,50,274,1764,13068,109584,\ldots$$

multinomial BellmanFord FloydWarshall TopoSort

6.3.3 Eulerian numbers

Number of permutations $\pi\in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j)>\pi(j+1)$, $k+1$ j :s s.t. $\pi(j)\geq j$, k j :s s.t. $\pi(j)>j$.

$$E(n,k)=(n-k)E(n-1,k-1)+(k+1)E(n-1,k)$$
$$E(n,0)=E(n,n-1)=1$$
$$E(n,k)=\sum_{j=0}^k(-1)^j\binom{n+1}{j}(k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k)=S(n-1,k-1)+kS(n-1,k)$$
$$S(n,1)=S(n,n)=1$$
$$S(n,k)=\frac{1}{k!}\sum_{j=0}^k(-1)^{k-j}\binom{k}{j}j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n)=1,1,2,5,15,52,203,877,4140,21147,\ldots$ For p prime,

$$B(p^m+n)\equiv mB(n)+B(n+1)\pmod{p}$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1n_2\cdots n_kn^{k-2}$
with degrees d_i : $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.7 Catalan numbers

$$C_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n+1}=\frac{(2n)!}{(n+1)n!}$$
$$C_0=1, \; C_{n+1}=\frac{2(2n+1)}{n+2}C_n, \; C_{n+1}=\sum C_iC_{n-i}$$

$$C_n=1,1,2,5,14,42,132,429,1430,4862,16796,58786,\ldots$$

- sub-diagonal monotone paths in an $n\times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2\max|w_i|<\sim 2^{63}$.
Time: $\mathcal{O}(VE)$

	830a8f, 23 lines
<pre>f5e const ll inf = LLONG_MAX; 556 struct Ed { int a, b, w, s() { return a < b ? a : -a; }}; 204 struct Node { ll dist = inf; int prev = -1; };</pre>	
<pre>019 void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) { ec0 nodes[s].dist = 0; 15a sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });</pre>	
<pre>96d int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices 900 rep(i,0,lim) for (Ed ed : eds) { c5c Node cur = nodes[ed.a], &dest = nodes[ed.b]; ed7 if (abs(cur.dist) == inf) continue; 4a6 ll d = cur.dist + ed.w; 167 if (d < dest.dist) { 729 dest.prev = ed.a; 68e dest.dist = (i < lim-1 ? d : -inf); 2e4 } cab }</pre>	
<pre>824 rep(i,0,lim) for (Ed e : eds) { 5e8 if (nodes[e.a].dist == -inf) 6d9 nodes[e.b].dist = -inf; 8d0 } 830 }</pre>	

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m , where $m[i][j]=\text{inf}$ if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j , inf if no path, or -inf if the path goes through a negative-weight cycle.
Time: $\mathcal{O}(N^3)$

	531245, 12 lines
<pre>964 const ll inf = 1LL << 62; 433 void floydWarshall(vector<vector<ll>>& m) { b0c int n = sz(m); 2b4 rep(i,0,n) m[i][i] = min(m[i][i], 0LL); 279 rep(k,0,n) rep(i,0,n) rep(j,0,n) 7be if (m[i][k] != inf && m[k][j] != inf) { 465 auto newDist = max(m[i][k] + m[k][j], -inf); 9a1 m[i][j] = min(m[i][j], newDist); 268 } 7f7 rep(k,0,n) if (m[k][k] < 0) rep(i,0,n) rep(j,0,n) 54f if (m[i][k] != inf && m[k][j] != inf) m[i][j] = -inf; 531 }</pre>	

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.
Time: $\mathcal{O}(|V|+|E|)$

	d678d8, 8 lines
<pre>01e vi topoSort(const vector<vi>& gr) { b73 vi indeg(sz(gr)), q; ed7 for (auto& li : gr) for (int x : li) indeg[x]++; 926 rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i); c7a rep(j,0,sz(q)) for (int x : gr[q[j]]) 2ca if (--indeg[x] == 0) q.push_back(x); 055 return q; d67 }</pre>	

7.2 Network flow

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}\left(V^2\sqrt{E}\right)$

```
01e1d4, 48 lines
49f struct PushRelabel {
a88     struct Edge {
815         int dest, back;
4b2         ll f, c;
d82     };
e68     vector<vector<Edge>> g;
74f     vector<ll> ec;
cf3     vector<Edge*> cur;
786     vector<vi> hs; vi H;
7fe     PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

b9c     void addEdge(int s, int t, ll cap, ll rcap=0) {
1f6         if (s == t) return;
d79         g[s].push_back({t, sz(g[t]), 0, cap});
c03         g[t].push_back({s, sz(g[s])-1, 0, rcap});
c17     }

774     void addFlow(Edge& e, ll f) {
e5e         Edge &back = g[e.dest][e.back];
5c8         if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
6d2         e.f += f; e.c -= f; ec[e.dest] += f;
3e9         back.f -= f; back.c += f; ec[back.dest] -= f;
b32     }
6cb     ll calc(int s, int t) {
5af         int v = sz(g); H[s] = v; ec[t] = 1;
22e         vi co(2*v); co[0] = v-1;
1c3         rep(i,0,v) cur[i] = g[i].data();
e82         for (Edge& e : g[s]) addFlow(e, e.c);

88a         for (int hi = 0;;) {
b9e             while (hs[hi].empty()) if (!hi--) return -ec[s];
04b             int u = hs[hi].back(); hs[hi].pop_back();
bfe             while (ec[u] > 0) // discharge u
0d1                 if (cur[u] == g[u].data() + sz(g[u])) {
57b                     H[u] = 1e9;
3de                     for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest
]+1)
fca                         H[u] = H[e.dest]+1, cur[u] = &e;
86b                         if (++co[H[u]],!--co[hi] && hi < v)
6a1                             rep(i,0,v) if (hi < H[i] && H[i] < v)
d60                                 --co[H[i]], H[i] = v + 1;
7bb                             hi = H[u];
8d8                         } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
0ed                             addFlow(*cur[u], min(ec[u], cur[u]->c));
d8e                         else ++cur[u];
10d                     }
68b                 }
f88                 bool leftOfMinCut(int a) { return H[a] >= sz(g); }
0ae             };
};
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

```
c4d #include <bits/extc++.h>

fe4 const ll INF = numeric_limits<ll>::max() / 4;

2fb struct MCMF {
5e6     struct edge {
```

```
070         int from, to, rev;
5ff         ll cap, cost, flow;
f64     };
4d0     int N;
dcd     vector<vector<edge>> ed;
9e3     vi seen;
8e1     vector<ll> dist, pi;
c0a     vector<edge*> par;

04b     MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

05d     void addEdge(int from, int to, ll cap, ll cost) {
75b         if (from == to) return;
e99         ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 }
);
3ec         ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0
});
76d     }

473     void path(int s) {
b25         fill(all(seen), 0);
922         fill(all(dist), INF);
e58         dist[s] = 0; ll di;

0c2         __gnu_pbds::priority_queue<pair<ll, int>> q;
9e3         vector<decltype(q)::point_iterator> its(N);
daa         q.push({ 0, s });

84e         while (!q.empty()) {
efe             s = q.top().second; q.pop();
9ff             seen[s] = 1; di = dist[s] + pi[s];
688             for (edge& e : ed[s]) if (!seen[e.to]) {
44b                 ll val = di - pi[e.to] + e.cost;
2c7                 if (e.cap - e.flow > 0 && val < dist[e.to]) {
fbf                     dist[e.to] = val;
a11                     par[e.to] = &e;
290                     if (its[e.to] == q.end())
233                         its[e.to] = q.push({ -dist[e.to], e.to });
ae4                     else
b79                         q.modify(its[e.to], { -dist[e.to], e.to });
801                 }
641             }
834         }
88d         rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
507     }

cac     pair<ll, ll> maxflow(int s, int t) {
142         ll totflow = 0, totcost = 0;
ba5         while (path(s), seen[t]) {
32d             ll fl = INF;
bdc             for (edge* x = par[t]; x; x = par[x->from])
83b                 fl = min(fl, x->cap - x->flow);

9e7             totflow += fl;
9d1             for (edge* x = par[t]; x; x = par[x->from]) {
6f3                 x->flow += fl;
adf                 ed[x->to][x->rev].flow -= fl;
810             }
375         }
38b         rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.
flow;
f00         return {totflow, totcost/2};
2b7     }

// If some costs can be negative, call this before
maxflow:
d16     void setpi(int s) { // (otherwise, leave this out)
927         fill(all(pi), INF); pi[s] = 0;
```

```
53b     int it = N, ch = 1; ll v;
f90     while (ch-- && it--)
f3a         rep(i,0,N) if (pi[i] != INF)
466             for (edge& e : ed[i]) if (e.cap)
780                 if ((v = pi[i] + e.cost) < pi[e.to])
223                     pi[e.to] = v, ch = 1;
0a2         assert(it >= 0); // negative cost cycle
8a4     }
583     };
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $\mathcal{O}(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.

```
482fe0, 36 lines
0e6 template<class T> T edmondsKarp(vector<unordered_map<int, T
>>&
676     graph, int source, int sink) {
dc8     assert(source != sink);
16a     T flow = 0;
324     vi par(sz(graph)), q = par;

6b3     for (;;) {
b68         fill(all(par), -1);
8ed         par[source] = 0;
f85         int ptr = 1;
968         q[0] = source;

481         rep(i,0,ptr) {
4df             int x = q[i];
0b6             for (auto e : graph[x]) {
47c                 if (par[e.first] == -1 && e.second > 0) {
edc                     par[e.first] = x;
7bf                     q[ptr++] = e.first;
3c9                     if (e.first == sink) goto out;
013                 }
e08             }
b22         }
b14         return flow;
a8b     out:
f9f         T inc = numeric_limits<T>::max();
ff7         for (int y = sink; y != source; y = par[y])
59b             inc = min(inc, graph[par[y]][y]);

b7f         flow += inc;
874         for (int y = sink; y != source; y = par[y]) {
734             int p = par[y];
39f             if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
634             graph[y][p] += inc;
868         }
98a     }
482 }
```

Dinic.h

Description: Flow algorithm with complexity $\mathcal{O}(VE\log U)$ where $U = \max|\text{cap}|$. $\mathcal{O}(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $\mathcal{O}(\sqrt{VE})$ for bipartite matching.

```
d7f0f1, 42 lines
14d struct Dinic {
923     struct Edge {
ca8         int to, rev;
ece         ll c, oc;
299         ll flow() { return max(oc - c, 0LL); } // if you need
flows
9d5     };
0aa     vi lvl, ptr, q;
31e     vector<vector<Edge>> adj;
fdd     Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
4f2     void addEdge(int a, int b, ll c, ll rcap = 0) {
```

```
3c8 adj[a].push_back({b, sz(adj[b]), c, c});
95d adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
a45 }
0e7 ll dfs(int v, int t, ll f) {
836 if (v == t || !f) return f;
241 for (int& i = ptr[v]; i < sz(adj[v]); i++) {
d70 Edge& e = adj[v][i];
591 if (lvl[e.to] == lvl[v] + 1)
fad if (ll p = dfs(e.to, t, min(f, e.c))) {
aed e.c -= p, adj[e.to][e.rev].c += p;
02f return p;
d3b }
f4f }
7da return 0;
720 }
e7b ll calc(int s, int t) {
219 ll flow = 0; q[0] = s;
b15 rep(L,0,31) do { // 'int L=30' maybe faster for random
data
df0 lvl = ptr = vi(sz(q));
5d9 int qi = 0, qe = lvl[s] = 1;
570 while (qi < qe && !lvl[t]) {
a7d int v = q[qi++];
8c4 for (Edge e : adj[v])
3c4 if (!lvl[e.to] && e.c >> (30 - L))
045 q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
16d }
12f while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
f27 } while (lvl[t]);
14d return flow;
2b9 }
761 bool leftOfMinCut(int a) { return lvl[a] != 0; }
d1f };
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.

Time: $\mathcal{O}(V^3)$

```
8b0e19, 21 lines
192 pair<int, vi> globalMinCut(vector<vi> mat) {
81f pair<int, vi> best = {INT_MAX, {}};
a4b int n = sz(mat);
165 vector<vi> co(n);
f64 rep(i,0,n) co[i] = {i};
a62 rep(ph,1,n) {
bfa vi w = mat[0];
6e3 size_t s = 0, t = 0;
76c rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio.
queue
c98 w[t] = INT_MIN;
2c2 s = t, t = max_element(all(w)) - w.begin();
9d9 rep(i,0,n) w[i] += mat[t][i];
8c0 }
727 best = min(best, {w[t] - mat[t][t], co[t]});
626 co[s].insert(co[s].end(), all(co[t]));
d24 rep(i,0,n) mat[s][i] += mat[t][i];
c7b rep(i,0,n) mat[i][s] = mat[s][i];
e25 mat[0][t] = INT_MIN;
076 }
6a6 return best;
8b0 };
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.

Time: $\mathcal{O}(V)$ Flow Computations

PushRelabel.h 0418b3, 13 lines

```
2d0 typedef array<ll, 3> Edge;
55d vector<Edge> gomoryHu(int N, vector<Edge> ed) {
ec4 vector<Edge> tree;
2ac vi par(N);
34e rep(i,1,N) {
0eb PushRelabel D(N); // Dinic also works
06d for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
531 tree.push_back({i, par[i], D.calc(i, par[i])});
9f8 rep(j,i+1,N)
ee9 if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
030 }
891 return tree;
041 };
```

7.3 Matching

HopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1 's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Time: $\mathcal{O}(E\sqrt{V})$

731cfb, 20 lines

```
674 int hopcroftKarp(vector<vi>& g, vi& r) {
c49 int n = sz(g), res = 0;
411 vi l(n, -1), q(n), d(n);
7c9 auto dfs = [&](auto f, int u) -> bool {
127 int t = exchange(d[u], 0) + 1;
457 for (int v : g[u])
5b7 if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
4cd return l[u] = v, r[v] = u, 1;
386 return 0;
f7a };
186 for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
983 rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
6cc rep(i,0,t) for (int v : g[q[i]]) {
0a8 if (r[v] == -1) f = 1;
cdd else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v]
];
714 }
612 if (!f) return res;
ea8 rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
19e }
731 };
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1 's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Usage: $vi\ btoa(m, -1); dfsMatching(g, btoa);$

Time: $\mathcal{O}(VE)$

522b98, 22 lines

```
a47 bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
98d if (btoa[j] == -1) return 1;
6aa vis[j] = 1; int di = btoa[j];
d09 for (int e : g[di])
400 if (!vis[e] && find(e, g, btoa, vis)) {
b1c btoa[e] = di;
107 return 1;
cc0 }
bf4 return 0;
d13 };
```

```
157 int dfsMatching(vector<vi>& g, vi& btoa) {
a61 vi vis;
49e rep(i,0,sz(g)) {
62e vis.assign(sz(btoa), 0);
0ed for (int j : g[i])
c46 if (find(j, g, btoa, vis)) {
407 btoa[j] = i;
5b1 break;
560 }
610 }
c95 return sz(btoa) - (int)count(all(btoa), -1);
522 };
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

DFSMatching.h da4196, 20 lines

```
0ba vi cover(vector<vi>& g, int n, int m) {
cb5 vi match(m, -1);
372 int res = dfsMatching(g, match);
b8f vector<bool> lfound(n, true), seen(m);
60f for (int it : match) if (it != -1) lfound[it] = false;
d5d vi q, cover;
3be rep(i,0,n) if (lfound[i]) q.push_back(i);
813 while (!q.empty()) {
e11 int i = q.back(); q.pop_back();
19f lfound[i] = 1;
113 for (int e : g[i]) if (!seen[e] && match[e] != -1) {
1ac seen[e] = true;
3b9 q.push_back(match[e]);
b97 }
b94 }
570 rep(i,0,n) if (!lfound[i]) cover.push_back(i);
a12 rep(i,0,m) if (seen[i]) cover.push_back(n+i);
8ed assert(sz(cover) == res);
630 return cover;
da4 };
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes $cost[N][M]$, where $cost[i][j]$ = cost for $L[i]$ to be matched with $R[j]$, and returns $(min\ cost, match)$, where $L[i]$ is matched with $R[match[i]]$. Negate costs for max cost. Requires $N \leq M$.

Time: $\mathcal{O}(N^2M)$

8f41c7, 24 lines

```
4fc template<class T>
e7c pair<T, vi> weightedMatching(vector<vector<T>> &C) {
1ec int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
0d6 vector<T> dist(m), pot(m);
176 vi match(i), rev(m, -1), cols(m), prev(m);
04f T d = 0, nd, cost = 0;
8c7 while (i--) {
131 rep(c,0,m) dist[c] = C[i][c], cols[c] = c, prev[c] = i;
8f2 for (s = 0;;) {
d1c rep(j,s,m) {
6f7 c = cols[j], nd = dist[c] - pot[c];
10c if (j == s || d > nd) d = nd, swap(cols[s], cols[j])
};
85c }
b75 if ((r = rev[c = cols[s++]]) == -1) break;
685 rep(j,0,m) if (dist[j] > (nd = C[r][j]-C[r][c]+dist[c]
)))
678 dist[j] = nd, prev[j] = r;
213 }
b2f cost += dist[c];
a16 while (s--) pot[cols[s]] = dist[cols[s]] - d;
e43 for (; r != i; swap(c, match[r]))
```

```
21f         r = rev[c] = prev[c];
2c5     }
d0a     return {cost, match};
8f4 }
```

GeneralMatching.h

Description: Matching for general graphs. Fails with probability N/mod .
Time: $\mathcal{O}(N^3)$

"../numerical/MatrixInverse-mod.h"	cb1912, 40 lines
<pre>376 vector<pii> generalMatching(int N, vector<pii>& ed) { 8d1 vector<vector<ll>> mat(N, vector<ll>(N)), A; ae1 for (pii pa : ed) { d77 int a = pa.first, b = pa.second, r = rand() % mod; 19e mat[a][b] = r, mat[b][a] = (mod - r) % mod; 614 } 576 int r = matInv(A = mat), M = 2*N - r, fi, fj; acf assert(r % 2 == 0); 9bc if (M != N) do { 480 mat.resize(M, vector<ll>(M)); dfa rep(i,0,N) { 159 mat[i].resize(M); ea9 rep(j,N,M) { d8f int r = rand() % mod; 60f mat[i][j] = r, mat[j][i] = (mod - r) % mod; 36a } be4 } c99 } while (matInv(A = mat) != M); 50a vi has(M, 1); vector<pii> ret; 17b rep(it,0,M/2) { 348 rep(i,0,M) if (has[i]) 2b3 rep(j,i+1,M) if (A[i][j] && mat[i][j]) { 493 fi = i; fj = j; goto done; be6 } assert(0); done: 1a9 if (fj < N) ret.emplace_back(fi, fj); fce has[fi] = has[fj] = 0; 3d1 rep(sw,0,2) { b06 ll a = modpow(A[fi][fj], mod-2); e24 rep(i,0,M) if (has[i] && A[i][fj]) { 600 ll b = A[i][fj] * a % mod; d78 rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod; 9de } 07f swap(fi,fj); 6c6 } f16 } cd0 return ret; cb1 }</pre>	

7.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.
Time: $\mathcal{O}(E + V)$

	76b5c9, 24 lines
<pre>508 vi val, comp, z, cont; c21 int Time, ncomps; d31 template<class G, class F> int dfs(int j, G& g, F& f) { 9b6 int low = val[j] = ++Time, x; z.push_back(j); ed2 for (auto e : g[j]) if (comp[e] < 0) 3cf low = min(low, val[e] ?: dfs(e,g,f));</pre>	

```
903     if (low == val[j]) {
12f         do {
b81             x = z.back(); z.pop_back();
c3d             comp[x] = ncomps;
6dd             cont.push_back(x);
cf1         } while (x != j);
122         f(cont); cont.clear();
d65         ncomps++;
6e1     }
862     return val[j] = low;
ab5 }
c74 template<class G, class F> void scc(G& g, F f) {
8d2     int n = sz(g);
46e     val.assign(n, 0); comp.assign(n, -1);
011     Time = ncomps = 0;
838     rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
76b }
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.
Usage: int eid = 0; ed.resize(N);
for each edge (a,b) {
ed[a].emplace_back(b, eid);
ed[b].emplace_back(a, eid++); }
bicomps([&](const vi& edgelist) {...});
Time: $\mathcal{O}(E + V)$

	c6b7c7, 32 lines
<pre>16a vi num, st; 5c7 vector<vector<pii>> ed; 5c1 int Time; bf2 template<class F> 3e8 int dfs(int at, int par, F& f) { 084 int me = num[at] = ++Time, top = me; ada for (auto [y, e] : ed[at]) if (e != par) { c29 if (num[y]) { 988 top = min(top, num[y]); 1af if (num[y] < me) 46b st.push_back(e); 978 } else { df9 int si = sz(st); cc9 int up = dfs(y, e, f); 21f top = min(top, up); ac6 if (up == me) { 772 st.push_back(e); fcf f(vi(st.begin() + si, st.end())); 71a st.resize(si); 35c } 14d else if (up < me) st.push_back(e); a3b else { /* e is a bridge */ } 9f2 } 746 } bf2 return top; fb9 } a32 template<class F> 6ac void bicomps(F f) { 11f num.assign(sz(ed), 0); ad5 rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f); c6b }</pre>	

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

	5f9706, 56 lines
<pre>d9d struct TwoSat { 257 int N; a0a vector<vi> gr; 7c0 vi values; // 0 = false, 1 = true c1f TwoSat(int n = 0) : N(n), gr(2*n) {} e10 int addVar() { // (optional) b4b gr.emplace_back(); ca3 gr.emplace_back(); 0f7 return N++; 8e7 } 144 void either(int f, int j) { 5e1 f = max(2*f, -1-2*f); bc6 j = max(2*j, -1-2*j); 7f8 gr[f].push_back(j^1); 511 gr[j].push_back(f^1); f60 } cbc void setValue(int x) { either(x, x); } 691 void atMostOne(const vi& li) { // (optional) 7d9 if (sz(li) <= 1) return; 772 int cur = ~li[0]; 66f rep(i,2,sz(li)) { 28a int next = addVar(); 3de either(cur, ~li[i]); 8bd either(cur, next); 001 either(~li[i], next); f47 cur = ~next; 7cd } f21 either(cur, ~li[1]); 069 } 594 vi val, comp, z; int time = 0; 923 int dfs(int i) { fa1 int low = val[i] = ++time, x; z.push_back(i); c93 for(int e : gr[i]) if (!comp[e]) c63 low = min(low, val[e] ?: dfs(e)); a0c if (low == val[i]) do { cf7 x = z.back(); z.pop_back(); 2c3 comp[x] = low; a8f if (values[x>>1] == -1) fb7 values[x>>1] = x&1; 5a0 } while (x != i); 3fe return val[i] = low; 088 } 126 bool solve() { 07c values.assign(N, -1); a75 val.assign(2*N, 0); comp = val; 27d rep(i,0,2*N) if (!comp[i]) dfs(i); a77 rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0; 95b return 1; 4fd } 5f9 };</pre>	

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.

Time: $\mathcal{O}(V + E)$

		780b64, 15 lines
87a	vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0)	
	{	
f0d	int n = sz(gr);	
ea6	vi D(n), its(n), eu(nedges), ret, s = {src};	
ddb	D[src]++; // to allow Euler paths, not just cycles	
fda	while (!s.empty()) {	
775	int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);	
7bb	if (it == end){ ret.push_back(x); s.pop_back(); continue	
	; }	
ad1	tie(y, e) = gr[x][it++];	
9e6	if (!eu[e]) {	
93b	D[x]--, D[y]++;	
fb2	eu[e] = 1; s.push_back(y);	
be6	}}	
605	for (int x : D) if (x < 0 sz(ret) != nedges+1) return {	
	};	
8b2	return {ret.rbegin(), ret.rend()};	
780	}	

7.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

Time: $\mathcal{O}(NM)$

		e210e2, 31 lines
f41	vi edgeColoring(int N, vector<pii> eds) {	
aa3	vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;	
04f	for (pii e : eds) ++cc[e.first], ++cc[e.second];	
a85	int u, v, ncols = *max_element(all(cc)) + 1;	
d26	vector<vi> adj(N, vi(ncols, -1));	
fc7	for (pii e : eds) {	
e80	tie(u, v) = e;	
123	fan[0] = v;	
2dd	loc.assign(ncols, 0);	
716	int at = u, end = u, d, c = free[u], ind = 0, i = 0;	
96c	while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)	
e45	loc[d] = ++ind, cc[ind] = d, fan[ind] = v;	
911	cc[loc[d]] = c;	
5a2	for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd	
	})	
8a9	swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);	
282	while (adj[fan[i]][d] != -1) {	
f3e	int left = fan[i], right = fan[++i], e = cc[i];	
e98	adj[u][e] = left;	
90b	adj[left][e] = u;	
4e1	adj[right][e] = -1;	
e70	free[right] = e;	
657	}	
a78	adj[u][d] = fan[i];	
2ee	adj[fan[i]][d] = u;	
783	for (int y : {fan[0], u, end})	
2b3	for (int& z = free[y] = 0; adj[y][z] != -1; z++);	
e9f	}	
967	rep(i,0,sz(eds))	
0c6	for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];	
ce6	return ret;	
e21	}	

7.6 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

		b0d5b1, 12 lines
753	typedef bitset<128> B;	
645	template<class F>	
05d	void cliques(vector& eds, F f, B P = ~B(), B X={}, B R={}	
	) {	
d46	if (!P.any()) { if (!X.any()) f(R); return; }	
abb	auto q = (P X)._Find_first();	
01a	auto cand = P & ~eds[q];	
876	rep(i,0,sz(eds)) if (cand[i]) {	
074	R[i] = 1;	
cf4	cliques(eds, f, P & eds[i], X & eds[i], R);	
c88	R[i] = P[i] = 0; X[i] = 1;	
2b8	}	
b0d	}	

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

		f7c0bc, 49 lines
54e	typedef vector<bitset<200>> vb;	
913	struct Maxclique {	
2b0	double limit=0.025, pk=0;	
93b	struct Vertex { int i, d=0; };	
b92	typedef vector<Vertex> vv;	
8ec	vb e;	
071	vv V;	
ccd	vector<vi> C;	
b54	vi qmax, q, S, old;	
f62	void init(vv& r) {	
4a8	for (auto& v : r) v.d = 0;	
993	for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];	
06b	sort(all(r), [](auto a, auto b) { return a.d > b.d; });	
16d	int mxD = r[0].d;	
964	rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;	
d5d	}	
e66	void expand(vv& R, int lev = 1) {	
ac1	S[lev] += S[lev - 1] - old[lev];	
860	old[lev] = S[lev - 1];	
67e	while (sz(R)) {	
09e	if (sz(q) + R.back().d <= sz(qmax)) return;	
20c	q.push_back(R.back().i);	
0b5	vv T;	
b0e	for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i	
	});	
e23	if (sz(T)) {	
c70	if (S[lev]++ / ++pk < limit) init(T);	
86a	int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1,	
	1);	
fb8	C[1].clear(), C[2].clear();	
abd	for (auto v : T) {	
d6b	int k = 1;	
3e1	auto f = [&](int i) { return e[v.i][i]; };	
6fc	while (any_of(all(C[k]), f)) k++;	
30a	if (k > mxk) mxk = k, C[mxk + 1].clear();	
f85	if (k < mnk) T[j++].i = v.i;	
8de	C[k].push_back(v.i);	
5eb	}	
df1	if (j > 0) T[j - 1].d = 0;	
bfc	rep(k,mnk,mxk + 1) for (int i : C[k])	
b4d	T[j].i = i, T[j++].d = k;	

e72	expand(T, lev + 1);
86a	} else if (sz(q) > sz(qmax)) qmax = q;
ad6	q.pop_back(), R.pop_back();
c01	}
901	}
12c	vi maxClique() { init(V), expand(V); return qmax; }
6c2	Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S)
	{
64b	rep(i,0,sz(e)) V.push_back({i});
21f	}
f7c	};

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertex-Cover.

7.7 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.

Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

		bfce85, 25 lines
750	vector<vi> treeJump(vi& P){	
d7f	int on = 1, d = 1;	
4e1	while (on < sz(P)) on *= 2, d++;	
401	vector<vi> jmp(d, P);	
bcb	rep(i,1,d) rep(j,0,sz(P))	
35d	jmp[i][j] = jmp[i-1][jmp[i-1][j]];	
9cd	return jmp;	
6d3	}	
d0c	int jmp(vector<vi>& tbl, int nod, int steps){	
68e	rep(i,0,sz(tbl))	
fa7	if (steps&(1<<i)) nod = tbl[i][nod];	
5f4	return nod;	
7ce	}	
48e	int lca(vector<vi>& tbl, vi& depth, int a, int b) {	
d4e	if (depth[a] < depth[b]) swap(a, b);	
afb	a = jmp(tbl, a, depth[a] - depth[b]);	
74e	if (a == b) return a;	
ea1	for (int i = sz(tbl); i--;) {	
67f	int c = tbl[i][a], d = tbl[i][b];	
653	if (c != d) a = c, b = d;	
863	}	
b79	return tbl[0][a];	
bfc	}	

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.

Time: $\mathcal{O}(N \log N + Q)$

		0f62fb, 21 lines
...	data-structures/RMQ.h"	
33e	struct LCA {	
818	int T = 0;	
a29	vi time, path, ret;	
169	RMQ<int> rmq;	
966	LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret))	
	{	
2de	void dfs(vector<vi>& C, int v, int par) {	
455	time[v] = T++;	
01f	for (int y : C[v]) if (y != par) {	
bdb	path.push_back(v), ret.push_back(time[v]);	
945	dfs(C, y, v);	

```
672     }
5ad   }

b01   int lca(int a, int b) {
055       if (a == b) return a;
61c       tie(a, b) = minmax(time[a], time[b]);
c24       return path[rmq.query(a, b)];
858   }

//dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}

0f6 };
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.

Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h"	9775a0, 21 lines
---------	------------------

```
6d4   typedef vector<pair<int, int>> vpi;
3b4   vpi compressTree(LCA& lca, const vi& subset) {
bb3       static vi rev; rev.resize(sz(lca.time));
53b       vi li = subset, &T = lca.time;
386       auto cmp = [&](int a, int b) { return T[a] < T[b]; };
6f0       sort(all(li), cmp);
485       int m = sz(li)-1;
60f       rep(i,0,m) {
dbb           int a = li[i], b = li[i+1];
bbc           li.push_back(lca.lca(a, b));
432       }
451       sort(all(li), cmp);
eab       li.erase(unique(all(li)), li.end());
1ce       rep(i,0,sz(li)) rev[li[i]] = i;
89f       vpi ret = {pii(0, li[0])};
649       rep(i,0,sz(li)-1) {
841           int a = li[i], b = li[i+1];
069           ret.emplace_back(rev[lca.lca(a, b)], b);
cef       }
2d9       return ret;
977   }
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.

Time: $\mathcal{O}((\log N)^2)$

"../data-structures/LazySegmentTree.h"	9547af, 46 lines
--	------------------

```
303   template <bool VALS_EDGES> struct HLD {
931       int N, tim = 0;
878       vector<vi> adj;
59d       vi par, siz, rt, pos;
431       Node *tree;
09c       HLD(vector<vi> adj_)
ac5           : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
8a0           rt(N), pos(N), tree(new Node(0, N)){ dfsSz(0); dfsHld(0)
; }
dfb   void dfsSz(int v) {
0cf       for (int& u : adj[v]) {
11a           adj[u].erase(find(all(adj[u]), v));
4c3           par[u] = v;
653           dfsSz(u);
867           siz[v] += siz[u];
897           if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
134       }
```

CompressTree HLD LinkCutTree LinkCutTree2

```
0e8   }
9a8   void dfsHld(int v) {
c7e       pos[v] = tim++;
001       for (int u : adj[v]) {
196           rt[u] = (u == adj[v][0] ? rt[v] : u);
211           dfsHld(u);
33f       }
be2   }
477   template <class B> void process(int u, int v, B op) {
c34       for (; v = par[rt[v]]) {
3ee           if (pos[u] > pos[v]) swap(u, v);
7f1           if (rt[u] == rt[v]) break;
a5d           op(pos[rt[v]], pos[v] + 1);
33e       }
856       op(pos[u] + VALS_EDGES, pos[v] + 1);
d77   }
4e5   void modifyPath(int u, int v, int val) {
056       process(u, v, [&](int l, int r) { tree->add(l, r, val);
});
fb3   }
d7f   int queryPath(int u, int v) { // Modify depending on
problem
a71       int res = -1e9;
1ae       process(u, v, [&](int l, int r) {
0d0           res = max(res, tree->query(l, r));
f84       });
cf6       return res;
7f9   }
ba8   int querySubtree(int v) { // modifySubtree is similar
fdd       return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v])
;
e96   }
954   };
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.

Time: All operations take amortized $\mathcal{O}(\log N)$.

	0fb462, 90 lines
--	------------------

```
bf2   struct Node { // Splay tree. Root's pp contains tree's
parent.
0dc       Node *p = 0, *pp = 0, *c[2];
038       bool flip = 0;
210       Node() { c[0] = c[1] = 0; fix(); }
a4e       void fix() {
5b7           if (c[0]) c[0]->p = this;
577           if (c[1]) c[1]->p = this;
// (+ update sum of subtree elements etc. if wanted)
426       }
34c       void pushFlip() {
1b9           if (!flip) return;
a0e           flip = 0; swap(c[0], c[1]);
da6           if (c[0]) c[0]->flip ^= 1;
168           if (c[1]) c[1]->flip ^= 1;
d94       }
829       int up() { return p ? p->c[1] == this : -1; }
b37       void rot(int i, int b) {
f8b           int h = i ^ b;
042           Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y :
x;
679           if ((y->p = p)) p->c[up()] = y;
59c           c[i] = z->c[i ^ 1];
9fc           if (b < 2) {
0ef               x->c[h] = y->c[h ^ 1];
17a               y->c[h ^ 1] = x;
653           }
3ed           z->c[i ^ 1] = this;
395           fix(); x->fix(); y->fix();
}
```

```
03a       if (p) p->fix();
8a0       swap(pp, y->pp);
966   }
74b   void splay() {
e2f       for (pushFlip(); p; ) {
7a5           if (p->p) p->p->pushFlip();
6ff           p->pushFlip(); pushFlip();
3ef           int c1 = up(), c2 = p->up();
7d3           if (c2 == -1) p->rot(c1, 2);
652           else p->p->rot(c2, c1 != c2);
1cf       }
5a4   }
d0e   Node* first() {
76d       pushFlip();
ca3       return c[0] ? c[0]->first() : (splay(), this);
e95   }
225   };

bea   struct LinkCut {
6c7       vector<Node> node;
47e       LinkCut(int N) : node(N) {}

391       void link(int u, int v) { // add an edge (u, v)
661           assert(!connected(u, v));
14e           makeRoot(&node[u]);
aee           node[u].pp = &node[v];
557       }
d8c       void cut(int u, int v) { // remove an edge (u, v)
612           Node *x = &node[u], *top = &node[v];
bdb           makeRoot(top); x->splay();
37b           assert(top == (x->pp ? x->c[0]));
33e           if (x->pp) x->pp = 0;
e75           else {
dec               x->c[0] = top->p = 0;
e4a               x->fix();
83e           }
47d       }
165       bool connected(int u, int v) { // are u, v in the same
tree?
f90           Node* nu = access(&node[u])->first();
760           return nu == access(&node[v])->first();
6fb       }
399       void makeRoot(Node* u) {
96c           access(u);
274           u->splay();
826           if (u->c[0]) {
4ee               u->c[0]->p = 0;
713               u->c[0]->flip ^= 1;
3ba               u->c[0]->pp = u;
e81               u->c[0] = 0;
248               u->fix();
9ee           }
9af       }
2bd       Node* access(Node* u) {
b7e           u->splay();
3cc           while (Node* pp = u->pp) {
9c5               pp->splay(); u->pp = 0;
9cf               if (pp->c[1]) {
f9b                   pp->c[1]->p = 0; pp->c[1]->pp = pp; }
9b8               pp->c[1] = u; pp->fix(); u = pp;
d19           }
81e           return u;
c39       }
0fb   };
```

LinkCutTree2.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree. It is 1-INDEXED!!!

Time: All operations take amortized $\mathcal{O}(\log N)$.

```
c6aa18, 105 lines
b8b struct SplayTree {
36e     struct Node {
3a5         int ch[2] = {0, 0}, p = 0;
0a0         ll self = 0, path = 0;           // Path aggregates
073         ll sub = 0, vir = 0;           // Subtree aggregates
c64         bool flip = 0;                // Lazy tags
e23     };
7e2     vector<Node> T;

ecd     SplayTree(int n) : T(n + 1) {}

840     void push(int x) {
b24         if (!x || !T[x].flip) return;
f12         int l = T[x].ch[0], r = T[x].ch[1];

9e8         T[l].flip ^= 1, T[r].flip ^= 1;
d24         swap(T[x].ch[0], T[x].ch[1]);
96d         T[x].flip = 0;
8fb     }

62d     void pull(int x) {
bcb         int l = T[x].ch[0], r = T[x].ch[1]; push(l); push(r);

c79         T[x].path = T[l].path + T[x].self + T[r].path;
280         T[x].sub = T[x].vir + T[l].sub + T[r].sub + T[x].self;
c43     }

8b1     void set(int x, int d, int y) {
fa8         T[x].ch[d] = y; T[y].p = x; pull(x);
449     }

dbc     void splay(int x) {
c88         auto dir = [&](int x) {
39a             int p = T[x].p; if (!p) return -1;
261             return T[p].ch[0] == x ? 0 : T[p].ch[1] == x ? 1 :
-1;
77e         };
cff         auto rotate = [&](int x) {
55d             int y = T[x].p, z = T[y].p, dx = dir(x), dy = dir(y)
;
0c9             set(y, dx, T[x].ch[!dx]);
4ec             set(x, !dx, y);
2bc             if (~dy) set(z, dy, x);
6d2             T[x].p = z;
15d         };
50e         for (push(x); ~dir(x); ) {
587             int y = T[x].p, z = T[y].p;
1b2             push(z); push(y); push(x);
1ac             int dx = dir(x), dy = dir(y);
f03             if (~dy) rotate(dx != dy ? x : y);
ada             rotate(x);
213         }
009     }
362 };

67e struct LinkCut : SplayTree {
e4e     LinkCut(int n) : SplayTree(n) {}

c41     int access(int x) {
48f         int u = x, v = 0;
837         for (; u; v = u, u = T[u].p) {
278             splay(u);
66b             int& ov = T[u].ch[1];
eb6             T[u].vir += T[ov].sub;
fcf             T[u].vir -= T[v].sub;
e5c             ov = v; pull(u);
2ed         }
```

```
fbf     return splay(x), v;
d69 }

c06     void reroot(int x) {
dbd         access(x); T[x].flip ^= 1; push(x);
213     }

46d     void Link(int u, int v) {
06a         reroot(u); access(v);
825         T[v].vir += T[u].sub;
524         T[u].p = v; pull(v);
04f     }

2bf     void Cut(int u, int v) {
ff4         reroot(u); access(v);
bd8         T[v].ch[0] = T[u].p = 0; pull(v);
706     }

// Rooted tree LCA. Returns 0 if u and v arent
// connected.

cd0     int LCA(int u, int v) {
7e5         if (u == v) return u;
7de         access(u); int ret = access(v);
e67         return T[u].p ? ret : 0;
c18     }

// Query subtree of u where v is outside the subtree.

886     ll Subtree(int u, int v) {
3d7         reroot(v); access(u); return T[u].vir + T[u].self;
c4c     }

// Query path [u..v]
bcf     ll Path(int u, int v) {
c07         reroot(u); access(v); return T[v].path;
b4e     }

// Update vertex u with value v
f51     void Update(int u, ll v) {
8c3         access(u); T[u].self += v; pull(u);
a49     }
c6a };

DirectedMST.h
Description: Finds a minimum spanning tree/arborescence of a directed
graph, given a root node. If no MST exists, returns -1.
Time:  $\mathcal{O}(E \log V)$ 
../data-structures/UnionFindRollback.h
39e620, 60 lines
030 struct Edge { int a, b; ll w; };
751 struct Node {
45a     Edge key;
348     Node *l, *r;
59f     ll delta;
958     void prop() {
c41         key.w += delta;
935         if (l) l->delta += delta;
69a         if (r) r->delta += delta;
cfc         delta = 0;
31f     }
61e     Edge top() { prop(); return key; }
677 };
d59 Node *merge(Node *a, Node *b) {
6b6     if (!a || !b) return a ? b;
839     a->prop(), b->prop();
7c5     if (a->key.w > b->key.w) swap(a, b);
c76     swap(a->l, (a->r = merge(b, a->r)));
046     return a;
5e3 }
821 void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }
```

```
ef4 pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
4a5     RollbackUF uf(n);
a73     vector<Node*> heap(n);
a4c     for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e})
;
5ec     ll res = 0;
9ed     vi seen(n, -1), path(n), par(n);
92e     seen[r] = r;
c7b     vector<Edge> Q(n), in(n, {-1,-1}), comp;
fc7     deque<tuple<int, int, vector<Edge>>> cys;
360     rep(s,0,n) {
96b         int u = s, qi = 0, w;
fae         while (seen[u] < 0) {
215             if (!heap[u]) return {-1,{};};
bcb             Edge e = heap[u]->top();
cc1             heap[u]->delta -= e.w, pop(heap[u]);
d9d             Q[qi] = e, path[qi++] = u, seen[u] = s;
fcb             res += e.w, u = uf.find(e.a);
e7e             if (seen[u] == s) {
2e1                 Node* cyc = 0;
516                 int end = qi, time = uf.time();
618                 do cyc = merge(cyc, heap[w = path[--qi]]);
7ce                 while (uf.join(u, w));
3eb                 u = uf.find(u), heap[u] = cyc, seen[u] = -1;
3a9                 cys.push_front({u, time, {&Q[qi], &Q[end]}});
ea7             }
db3         }
930         rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
f2b     }

186     for (auto& [u,t,comp] : cys) { // restore sol (optional)
55d         uf.rollback(t);
6dd         Edge inEdge = in[u];
a32         for (auto& e : comp) in[uf.find(e.b)] = e;
b09         in[uf.find(inEdge.b)] = inEdge;
c5d     }
4f8     rep(i,0,n) par[i] = in[i].a;
d28     return {res, par};
39e }
```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $\text{mat}[a][b]--$, $\text{mat}[b][b]++$ (and $\text{mat}[b][a]--$, $\text{mat}[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

		47ec0a, 28 lines
48b	<pre>template <class T> int sgn(T x) { return (x > 0) - (x < 0); }</pre>	
ref	<pre>template<class T></pre>	
742	<pre>struct Point {</pre>	
f77	<pre> typedef Point P;</pre>	
fa7	<pre> T x, y;</pre>	
551	<pre> explicit Point(T x=0, T y=0) : x(x), y(y) {}</pre>	
1a0	<pre> bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y) ; }</pre>	
3a2	<pre> bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y) ; }</pre>	
1dc	<pre> P operator+(P p) const { return P(x+p.x, y+p.y); }</pre>	
189	<pre> P operator-(P p) const { return P(x-p.x, y-p.y); }</pre>	
268	<pre> P operator*(T d) const { return P(x*d, y*d); }</pre>	
8cb	<pre> P operator/(T d) const { return P(x/d, y/d); }</pre>	
716	<pre> T dot(P p) const { return x*p.x + y*p.y; }</pre>	
7ec	<pre> T cross(P p) const { return x*p.y - y*p.x; }</pre>	
520	<pre> T cross(P a, P b) const { return (a-*this).cross(b-*this); }</pre>	
e7b	<pre> T dist2() const { return x*x + y*y; }</pre>	
039	<pre> double dist() const { return sqrt((double)dist2()); } // angle to x-axis in interval [-pi, pi]</pre>	
cc7	<pre> double angle() const { return atan2(y, x); }</pre>	
b02	<pre> P unit() const { return *this/dist(); } // makes dist()==1</pre>	
e05	<pre> P perp() const { return P(-y, x); } // rotates +90 degrees</pre>	
c0e	<pre> P normal() const { return perp().unit(); } // returns point rotated 'a' radians ccw around the origin</pre>	
91d	<pre> P rotate(double a) const {</pre>	
e45	<pre> return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }</pre>	
706	<pre> friend ostream& operator<<(ostream& os, P p) {</pre>	
0e4	<pre> return os << "(" << p.x << ", " << p.y << ")"; }</pre>	
47e	<pre>};</pre>	

lineDistance.h

Description:

Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

"Point.h"	53630a, 4 lines
7dc	<pre>template<class P></pre>
02b	<pre>double lineDist(P a, P b, P p) {</pre>
251	<pre> return (double) (b-a).cross(p-a)/(b-a).dist();</pre>
536	<pre>}</pre>

SegmentDistance.h

Description:

Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2,2), p(1,1);

bool onSegment = segDist(a,b,p) < 1e-10;

"Point.h"	d5f39f, 6 lines
626	<pre>typedef Point<double> P;</pre>
39b	<pre>double segDist(P s, P e, P p) {</pre>
fe1	<pre> if (s==e) return (p-s).dist();</pre>
227	<pre> auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));</pre>

```
717     return ((p-s)*d-(e-s)*t).dist()/d;  
d5f }
```

SegmentIntersection.h

Description:

If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1,e1,s2,e2);

		9d57f2, 13 lines
dae	<pre>template<class P> vector<P> segInter(P a, P b, P c, P d) {</pre>	
f4c	<pre> auto oa = c.cross(d, a), ob = c.cross(d, b),</pre>	
504	<pre> oc = a.cross(b, c), od = a.cross(b, d); // Checks if intersection is single non-endpoint point.</pre>	
dec	<pre> if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)</pre>	
ab1	<pre> return {(a * ob - b * oa) / (ob - oa)};</pre>	
431	<pre> set<P> s;</pre>	
d73	<pre> if (onSegment(c, d, a)) s.insert(a);</pre>	
9f9	<pre> if (onSegment(c, d, b)) s.insert(b);</pre>	
64d	<pre> if (onSegment(a, b, c)) s.insert(c);</pre>	
1dc	<pre> if (onSegment(a, b, d)) s.insert(d);</pre>	
c50	<pre> return {all(s)};</pre>	
9d5	<pre>}</pre>	

lineIntersection.h

Description:

If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

Usage: auto res = lineInter(s1,e1,s2,e2);

		a01f81, 8 lines
"Point.h"		
7dc	<pre>template<class P></pre>	
ebe	<pre>pair<int, P> lineInter(P s1, P e1, P s2, P e2) {</pre>	
662	<pre> auto d = (e1 - s1).cross(e2 - s2);</pre>	
a6b	<pre> if (d == 0) // if parallel</pre>	
47e	<pre> return {-(s1.cross(e1, s2) == 0), P(0, 0)};</pre>	
dfc	<pre> auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);</pre>	
c4c	<pre> return {1, (s1 * p + e1 * q) / d};</pre>	
a01	<pre>}</pre>	

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;

		ca4665, 9 lines
"Point.h"		
7dc	<pre>template<class P></pre>	
fad	<pre>int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }</pre>	
bb2	<pre>template<class P></pre>	
6d4	<pre>int sideOf(P s, P e, P p, double eps) {</pre>	
914	<pre> auto a = (e-s).cross(p-s);</pre>	

```
bf3     double l = (e-s).dist()*eps;  
c18     return (a > l) - (a < -l);  
ca4 }
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p) <=epsilon) instead when using Point<double>.

"Point.h"	c597e8, 3 lines
514	<pre>template<class P> bool onSegment(P s, P e, P p) {</pre>
b95	<pre> return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;</pre>
c59	<pre>}</pre>

linearTransformation.h

Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

		7871e9, 5 lines
"Point.h"		
626	<pre>typedef Point<double> P;</pre>	
a92	<pre>P linearTransformation(P p0, P p1, P q0, P q1, P r) {</pre>	
9f8	<pre> P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));</pre>	
4bc	<pre> return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2()</pre>	
	<pre>);</pre>	
787	<pre>}</pre>	

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

		b5562d, 5 lines
"Point.h"		
7dc	<pre>template<class P></pre>	
31a	<pre>P lineProj(P a, P b, P p, bool refl=false) {</pre>	
3c6	<pre> P v = b - a;</pre>	
3d9	<pre> return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();</pre>	
b55	<pre>}</pre>	

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

		0f0602, 35 lines
755	<pre>struct Angle {</pre>	
022	<pre> int x, y;</pre>	
76e	<pre> int t;</pre>	
d18	<pre> Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}</pre>	
6c9	<pre> Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }</pre>	
020	<pre> int half() const {</pre>	
b0d	<pre> assert(x y);</pre>	
9d5	<pre> return y < 0 (y == 0 && x < 0);</pre>	
39c	<pre> }</pre>	
12a	<pre> Angle t90() const { return {-y, x, t + (half() && x >= 0)} ; }</pre>	
05c	<pre> Angle t180() const { return {-x, -y, t + half()}; }</pre>	
3dd	<pre> Angle t360() const { return {x, y, t + 1}; }</pre>	
e25	<pre>};</pre>	
c1e	<pre>bool operator<(Angle a, Angle b) { // add a.dist2() and b.dist2() to also compare distances</pre>	
a1f	<pre> return make_tuple(a.t, a.half(), a.y * (ll)b.x) <</pre>	
7d3	<pre> make_tuple(b.t, b.half(), a.x * (ll)b.y);</pre>	
e78	<pre>}</pre>	

```
// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
ccb pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
48d   if (b < a) swap(a, b);
c03   return (b < a.tl80() ?
408       make_pair(a, b) : make_pair(b, a.t360()));
ecc }
c11 Angle operator+(Angle a, Angle b) { // point a + vector b
c7f   Angle r(a.x + b.x, a.y + b.y, a.t);
7cc   if (a.tl80() < r) r.t--;
e12   return r.tl80() < a ? r.t360() : r;
3fb }
89a Angle angleDiff(Angle a, Angle b) { // angle b - angle a
99d   int tu = b.t - a.t; a.t = b.t;
33f   return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)
};
oF0 }
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h"	84d6d3, 11 lines
<pre>626 typedef Point<double> P; 888 bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) { 7e5 if (a == b) { assert(r1 != r2); return false; } 2e6 P vec = b - a; deb double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2, 7b2 p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p* d2; 6ad if (sum*sum < d2 dif*dif > d2) return false; 70d P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2); 3dd *out = {mid + per, mid - per}; 212 return true; 84d }</pre>	

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h"	b0153d, 13 lines
<pre>7dc template<class P> e80 vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) { c7e P d = c2 - c1; 45b double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr; c18 if (d2 == 0 h2 < 0) return {}; f9f vector<pair<P, P>> out; 007 for (double sign : {-1, 1}) { 48b P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2; 729 out.push_back({c1 + v * r1, c2 + v * r2}); 41b } 231 if (h2 == 0) out.pop_back(); 054 return out; b01 }</pre>	

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h"	e0cfba, 9 lines
-----------	-----------------

```
7dc template<class P>
040 vector<P> circleLine(P c, double r, P a, P b) {
cdd   P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
e51   double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
64a   if (h2 < 0) return {};
3d9   if (h2 == 0) return {p};
1be   P h = ab.unit() * sqrt(h2);
3b1   return {p - h, p + h};
e0c }
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

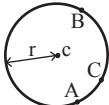
Time: $\mathcal{O}(n)$

"../content/geometry/Point.h"	19add1, 19 lines
<pre>626 typedef Point<double> P; cf6 #define arg(p, q) atan2(p.cross(q), p.dot(q)) cf0 double circlePoly(P c, double r, vector<P> ps) { 419 auto tri = [&](P p, P q) { a6c auto r2 = r * r / 2; c04 P d = q - p; 702 auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2 ()); 4c3 auto det = a * a - b; 371 if (det <= 0) return arg(p, q) * r2; 15e auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det)); 1b0 if (t < 0 1 <= s) return arg(p, q) * r2; c77 P u = p + d * s, v = q + d * (t-1); 3a1 return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2; e30 }; 706 auto sum = 0.0; b70 rep(i,0,sz(ps)) 78a sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c); 526 return sum; 19a }</pre>	

circumcircle.h

Description:

The circumcirlce of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



"Point.h"	6cffc5, 9 lines
<pre>626 typedef Point<double> P; de8 double ccRadius(P A, P B, P C) { 068 return (B-A).dist()*(C-B).dist()*(A-C).dist() / f6b abs((B-A).cross(C-A)) / 2; d0b } 3c9 P ccCenter(P A, P B, P C) { ee8 P b = C-A, c = B-A; 227 return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2; 6cf }</pre>	

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points. **Time:** expected $\mathcal{O}(n)$

"circumcircle.h"	09dd0a, 17 lines
<pre>a28 pair<P, double> mec(vector<P> ps) { 31f shuffle(all(ps), mt19937(time(0))); 76d P o = ps[0]; 56a double r = 0, EPS = 1 + 1e-8; b50 rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) { 5e7 o = ps[i], r = 0; af7 rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) { 57d o = (ps[i] + ps[j]) / 2; da0 r = (o - ps[i]).dist();</pre>	

```
14c       rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
931           o = ccCenter(ps[i], ps[j], ps[k]);
b9c           r = (o - ps[i]).dist();
7cd       }
03d   }
bfa   }
5eb   return {o, r};
09d }
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}}; bool in = inPolygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h"	2bf504, 11 lines
<pre>7dc template<class P> 8cf bool inPolygon(vector<P> &p, P a, bool strict = true) { 68a int cnt = 0, n = sz(p); 49a rep(i,0,n) { 1c1 P q = p[(i + 1) % n]; ca7 if (onSegment(p[i], q, a)) return !strict; //or: if (segDist(p[i], q, a) <= eps) return !strict; 8d1 cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0; ae1 } 3f2 return cnt; 2bf }</pre>	

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"	f12300, 6 lines
<pre>4fc template<class T> df7 T polygonArea2(vector<Point<T>>& v) { ab8 T a = v.back().cross(v[0]); 071 rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]); b19 return a; f12 }</pre>	

PolygonCenter.h

Description: Returns the center of mass for a polygon. **Time:** $\mathcal{O}(n)$

"Point.h"	9706dc, 9 lines
<pre>626 typedef Point<double> P; fa2 P polygonCenter(const vector<P>& v) { a6f P res(0, 0); double A = 0; 1dc for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) { 082 res = res + (v[i] + v[j]) * v[j].cross(v[i]); c6e A += v[j].cross(v[i]); 017 } 9d5 return res / A / 3; 970 }</pre>	

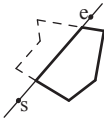
PolygonCut.h

Description:

Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...; p = polygonCut(p, P(0,0), P(1,0));

"Point.h"	d07181, 13 lines
<pre>626 typedef Point<double> P; b4b vector<P> polygonCut(const vector<P>& poly, P s, P e) { b83 vector<P> res;</pre>	



```
f63 rep(i,0,sz(poly)) {
366     P cur = poly[i], prev = i ? poly[i-1] : poly.back();
f68     auto a = s.cross(e, cur), b = s.cross(e, prev);
fa5     if ((a < 0) != (b < 0))
641         res.push_back(cur + (prev - cur) * (a / (a - b)));
0c6     if (a < 0)
b8d         res.push_back(cur);
354 }
b7d return res;
d07 }
```

PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)

Time: $\mathcal{O}(N^2)$, where N is the total number of points

"Point.h", "sideOf.h"	3931c6, 33 lines
-----------------------	------------------

```
626 typedef Point<double> P;
940 double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y;
    }
51e double polyUnion(vector<vector<P>>& poly) {
968     double ret = 0;
49c     rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
1ea         P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
e9d         vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
aea         rep(j,0,sz(poly)) if (i != j) {
036             rep(u,0,sz(poly[j])) {
082                 P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])
    ];
c62         int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
ac8         if (sc != sd) {
a48             double sa = C.cross(D, A), sb = C.cross(D, B);
aea             if (min(sc, sd) < 0)
13f                 segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
ce5             } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))
    >0){
a46                 segs.emplace_back(rat(C - A, B - A), 1);
d44                 segs.emplace_back(rat(D - A, B - A), -1);
675             }
c4b         }
a19     }
97a     sort(all(segs));
4e8     for (auto& s : segs) s.first = min(max(s.first, 0.0),
1.0);
00b     double sum = 0;
40a     int cnt = segs[0].second;
317     rep(j,1,sz(segs)) {
84a         if (!cnt) sum += segs[j].first - segs[j - 1].first;
625         cnt += segs[j].second;
d33     }
0e3     ret += A.cross(B) * sum;
6f2 }
52e return ret / 2;
393 }
```

ConvexHull.h

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

"Point.h"	310954, 13 lines
-----------	------------------

```
2c0 typedef Point<ll> P;
af1 vector<P> convexHull(vector<P> pts) {
bf0     if (sz(pts) <= 1) return pts;
086     sort(all(pts));
3e3     vector<P> h(sz(pts)+1);
```



```
cc9 int s = 0, t = 0;
8b7 for (int it = 2; it--; s = --t, reverse(all(pts)))
2fd     for (P p : pts) {
e7e         while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t
--;
f4a         h[t++] = p;
56a     }
b08     return {h.begin(), h.begin() + t - (t == 2 && h[0] == h
    [1])};
310 }
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Time: $\mathcal{O}(n)$

"Point.h"	c571b8, 12 lines
-----------	------------------

```
2c0 typedef Point<ll> P;
28b array<P, 2> hullDiameter(vector<P> S) {
9bd     int n = sz(S), j = n < 2 ? 0 : 1;
12e     pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
5c7     rep(i,0,j)
e5f         for (; j = (j + 1) % n) {
263             res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
e7f             if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >=
    0)
49f                 break;
cf8         }
d9b     return res.second;
c57 }
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h"	71446b, 14 lines
--------------------------------------	------------------

```
2c0 typedef Point<ll> P;
912 bool inHull(const vector<P>& l, P p, bool strict = true) {
3f3     int a = 1, b = sz(l) - 1, r = !strict;
7a3     if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
b8c     if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
3c3     if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <=
-r)
bc8         return false;
709     while (abs(a - b) > 1) {
e79         int c = (a + b) / 2;
2a9         (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
e4f     }
0b5     return sgn(l[a].cross(l[b], p)) < r;
714 }
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

"Point.h"	7cf45b, 39 lines
-----------	------------------

```
530 #define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n
    ]))
d4b #define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
838 template <class P> int extrVertex(vector<P>& poly, P dir) {
```

```
6c6 int n = sz(poly), lo = 0, hi = n;
b9d if (extr(0)) return 0;
b3e while (lo + 1 < hi) {
407     int m = (lo + hi) / 2;
1b2     if (extr(m)) return m;
604     int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
c73     (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) =
    m;
efd }
743 return lo;
ba4 }
```

```
911 #define cmpL(i) sgn(a.cross(poly[i], b))
26a template <class P>
d01 array<int, 2> lineHull(P a, P b, vector<P>& poly) {
d0d     int endA = extrVertex(poly, (a - b).perp());
bc5     int endB = extrVertex(poly, (b - a).perp());
ff7     if (cmpL(endA) < 0 || cmpL(endB) > 0)
07b         return {-1, -1};
a8a     array<int, 2> res;
aa6     rep(i,0,2) {
090         int lo = endB, hi = endA, n = sz(poly);
0ef         while ((lo + 1) % n != hi) {
710             int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
d0c             (cmpL(m) == cmpL(endB) ? lo : hi) = m;
72e         }
c0e         res[i] = (lo + !cmpL(hi)) % n;
541         swap(endA, endB);
d56     }
d84     if (res[0] == res[1]) return {res[0], -1};
e14     if (!cmpL(res[0]) && !cmpL(res[1]))
5b4         switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
ab4             case 0: return {res[0], res[0]};
e5b             case 2: return {res[1], res[1]};
54f         }
cba     return res;
7cf }
```

MinkowskiSum.h

Description: Computes the Minkowski sum C of two convex polygons A and B . It is proven that C is a convex polygon with at most $|A| + |B|$ vertices. Vertices of polygons A and B must be given in CCW order, and vertices of C will be returned in CCW order, similarly.

Time: $\mathcal{O}(|A| + |B|)$

"Point.h"	f85ee8, 23 lines
-----------	------------------

```
2c0 typedef Point<ll> P;
e5a void reorder_polygon(vector<P>& a) {
68a     int p = 0;
af6     rep(i, 1, sz(a)) {
18d         if (tie(a[i].y, a[i].x) < tie(a[p].y, a[p].x)) p = i;
c93     }
592     rotate(a.begin(), a.begin() + p, a.end());
d64 }
124 vector<P> minkowski(vector<P> a, vector<P> b) {
6f1     reorder_polygon(a);
f52     reorder_polygon(b);
46c     a.push_back(a[0]); a.push_back(a[1]);
023     b.push_back(b[0]); b.push_back(b[1]);
602     vector<P> c;
8e7     int i = 0, j = 0;
37f     while (i < sz(a) - 2 || j < sz(b) - 2){
d96         c.push_back(a[i] + b[j]);
777         auto cross = {a[i + 1] - a[i]}.cross(b[j + 1] - b[j]);
cd8         if (cross >= 0 && i < sz(a) - 2) ++i;
b79         if (cross <= 0 && j < sz(b) - 2) ++j;
a01     }
4cc     return c;
f85 }
```


8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

"Point.h"	ac41a6, 17 lines
2c0 typedef Point<ll> P;	
754 pair<P, P> closest(vector<P> v) {	
b02 assert(sz(v) > 1);	
8f0 set<P> S;	
9e7 sort(all(v), [](P a, P b) { return a.y < b.y; });	
db6 pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};	
2ac int j = 0;	
14a for (P p : v) {	
484 P d{1 + (ll)sqrt(ret.first), 0};	
0a3 while (v[j].y <= p.y - d.x) S.erase(v[j++]);	
270 auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d	
)	
e75 for (; lo != hi; ++lo)	
412 ret = min(ret, {(*lo - p).dist2(), { *lo, p } });	
atb S.insert(p);	
a43 }	
65a return ret.second;	
ac4 }	

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights $w(p, q) = -p.x - q.x + -p.y - q.y$. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.

Time: $\mathcal{O}(N \log N)$

"Point.h"	df6f59, 23 lines
bbe typedef Point<int> P;	
107 vector<array< int , 3>> manhattanMST(vector<P> ps) {	
82b vi id(sz(ps));	
129 iota(all(id), 0);	
bde vector<array< int , 3>> edges;	
463 rep(k,0,4) {	
55b sort(all(id), [&](int i, int j) {	
f00 return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});	
0a2 map< int , int > sweep;	
6ad for (int i : id) {	
232 for (auto it = sweep.lower_bound(-ps[i].y);	
734 it != sweep.end(); sweep.erase(it++)) {	
931 int j = it->second;	
529 P d = ps[i] - ps[j];	
874 if (d.y > d.x) break ;	
5f4 edges.push_back({d.y + d.x, i, j});	
28e }	
5f0 sweep[-ps[i].y] = i;	
9ea }	
9c2 for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.	
y);	
666 }	
af3 return edges;	
df6 }	

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h"	69ff55, 63 lines
9a6 typedef long long T;	
d3d typedef Point<T> P;	
3b6 const T INF = numeric_limits<T>::max();	
f3f bool on_x(P a, P b) { return a.x < b.x; }	
e34 bool on_y(P a, P b) { return a.y < b.y; }	
ff9 struct Node {	

3c2 P pt; <i>// if this is a leaf, the single point in it</i>	
397 T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; <i>// bounds</i>	
975 Node *first = 0, *second = 0;	
457 T distance(P p) { <i>// min squared distance to a point</i>	
c8f T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);	
27f T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);	
036 return (P(x,y) - p).dist2();	
d9f }	
d27 Node(vector<P>&& vp) : pt(vp[0]) {	
455 for (P p : vp) {	
d41 x0 = min(x0, p.x); x1 = max(x1, p.x);	
d62 y0 = min(y0, p.y); y1 = max(y1, p.y);	
d96 }	
6a8 if (vp.size() > 1) {	
002 <i>// split on x if width >= height (not ideal...)</i>	
sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);	
<i>// divide by taking half the array for each child (</i>	
<i>not</i>	
<i>// best performance with many duplicates in the</i>	
<i>middle)</i>	
3f2 int half = sz(vp)/2;	
849 first = new Node({vp.begin(), vp.begin() + half});	
904 second = new Node({vp.begin() + half, vp.end()});	
a6c }	
16b }	
980 };	
d0a struct KDTree {	
14d Node* root;	
50e KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}	
d19 pair<T, P> search(Node *node, P p) {	
7a4 if (!node->first) {	
<i>// uncomment if we should not find the point itself:</i>	
<i>// if (p == node->pt) return {INF, P()};</i>	
d2d return make_pair((p - node->pt).dist2(), node->pt);	
5e2 }	
9e7 Node *f = node->first, *s = node->second;	
bbc T bfirst = f->distance(p), bsec = s->distance(p);	
a3f if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);	
<i>// search closest side first, other side if needed</i>	
76c auto best = search(f, p);	
e11 if (bsec < best.first)	
271 best = min(best, search(s, p));	
a13 return best;	
4c8 }	

<i>// find nearest point to a point, and its squared</i>	
<i>distance</i>	
<i>// (requires an arbitrary operator< for Point)</i>	
86f pair<T, P> nearest(P p) {	
727 return search(root, p);	
056 }	
69f };	

DelaunayTriangulation.h

Description: Computes the Delaunay triangulation of a set of points. Each circumcircle contains none of the input points. If any three points are collinear or any four are on the same circle, behavior is undefined.

Time: $\mathcal{O}(n^2)$

"Point.h", "3dHull.h"	c0e7bc, 10 lines
6ab template<class P, class F>	
b5f void delaunay(vector<P>& ps, F trifun) {	
6b1 if (sz(ps) == 3) { int d = (ps[0].cross(ps[1], ps[2]) < 0)	
;	

0c9 trifun(0,1+d,2-d); }	
d1e vector<P3> p3;	
3ff for (P p : ps) p3.emplace_back(p.x, p.y, p.dist2());	
263 if (sz(ps) > 3) for (auto t:hull13d(p3)) if ((p3[t.b]-p3[t.a	
]);	
cf3 cross(p3[t.c]-p3[t.a]).dot(P3(0,0,1)) < 0)	
c20 trifun(t.a, t.c, t.b);	
c0e }	

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ... }, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

"Point.h"	eefdf5, 88 lines
2c0 typedef Point<ll> P;	
c41 typedef struct Quad* Q;	
56a typedef __int128_t ll1; <i>// (can be ll if coords are < 2e4)</i>	
503 P arb(LLONG_MAX,LLONG_MAX); <i>// not equal to any other point</i>	
4dc struct Quad {	
3fb Q rot, o; P p = arb; bool mark;	
001 P& F() { return r()->p; }	
9a9 Q& r() { return rot->rot; }	
a6d Q prev() { return rot->o->rot; }	
c2b Q next() { return r()->prev(); }	
aa9 } *H;	
a9b bool circ(P p, P a, P b, P c) { <i>// is p in the circumcircle?</i>	
402 ll1 p2 = p.dist2(), A = a.dist2()-p2,	
4e3 B = b.dist2()-p2, C = c.dist2()-p2;	
ba6 return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B >	
0;	
cae }	
ced Q makeEdge(P orig, P dest) {	
ee2 Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};	
603 H = r->o; r->r()->r() = r;	
224 rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r	
();	
e2e r->p = orig; r->F() = dest;	
9e2 return r;	
25c }	
ffa void splice(Q a, Q b) {	
6a5 swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);	
0ef }	
2b1 Q connect(Q a, Q b) {	
1b0 Q q = makeEdge(a->F(), b->p);	
bbc splice(q, a->next());	
30d splice(q->r(), b);	
f47 return q;	
eeF }	

b16 pair<Q,Q> rec(const vector<P>& s) {	
153 if (sz(s) <= 3) {	
9e4 Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back())	
;	
d46 if (sz(s) == 2) return { a, a->r() };	
f14 splice(a->r(), b);	
544 auto side = s[0].cross(s[1], s[2]);	
d0b Q c = side ? connect(b, a) : 0;	
d0c return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };	
60c }	

789 #define H(e) e->F(), e->p	
0aa #define valid(e) (e->F().cross(H(base)) > 0)	
9a0 Q A, B, ra, rb;	
4db int half = sz(s) / 2;	

```
576 tie(ra, A) = rec({all(s) - half});
091 tie(B, rb) = rec({sz(s) - half + all(s)});
fe1 while ((B->p.cross(H(A)) < 0 && (A = A->next()) ||
e9b (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
ad8 Q base = connect(B->r(), A);
ad8 if (A->p == ra->p) ra = base->r();
e3b if (B->p == rb->p) rb = base;

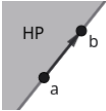
880 #define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
5b7 while (circ(e->dir->F(), H(base), e->F())) { \
e56 Q t = e->dir; \
8f7 splice(e, e->prev()); \
e15 splice(e->r(), e->r()->prev()); \
1ea e->o = H; H = e; e = t; \
d41 }
d06 for (;;) {
9b3 DEL(LC, base->r(), o); DEL(RC, base, prev());
0bd if (!valid(LC) && !valid(RC)) break;
4af if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
cf4 base = connect(RC, base->r());
f43 else
4e9 base = connect(base->r(), LC->r());
17c }
505 return { ra, rb };
2d9 }
```

```
198 vector<P> triangulate(vector<P> pts) {
f8e sort(all(pts)); assert(unique(all(pts)) == pts.end());
af7 if (sz(pts) < 2) return {};
d74 Q e = rec(pts).first;
35c vector<Q> q = {e};
51f int qi = 0;
19b while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
000 #define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p)
; \
d69 q.push_back(c->r()); c = c->next(); } while (c != e); }
8e3 ADD; pts.clear();
fd8 while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
8cd return pts;
eef }
```

HalfPlaneIntersection.h

Description:

Computes the intersection of n half-planes. Each half-plane is represented as the region to the left side of a vector $\overrightarrow{ab}$. If the half-planes are already given sorted by angle, the sort in the first line of hpi can be removed, making the time $O(n)$.



Time: $O(n \log n)$

"Point.h"	99c1df, 43 lines
-----------	------------------

```
236 typedef long double dbl;
534 typedef Point<dbl> P;
4b0 const dbl eps = 1e-9;
1f2 struct HP {
13c P p, pq;
231 dbl ang;
6c4 HP() {}
7fc HP(const P& a, const P& b): p(a), pq(b - a) {
1ab ang = atan2l(pq.y, pq.x);
fd4 }
bb4 bool out(const P& r) { return pq.cross(r - p) < -eps; }
84f bool operator<(const HP& e) const { return ang < e.ang; }
084 friend P inter(const HP& s, const HP& t) {
f12 dbl alpha = (t.p-s.p).cross(t.pq) / s.pq.cross(t.pq);
d48 return s.p + (s.pq * alpha);
411 }
582 };
a06 vector<P> hpi(vector<HP>& H) {
```

```
279 sort(all(H)); // can be removed if HPs are already sorted
a75 deque<HP> dq;
e04 int len = 0;
462 rep(i, 0, sz(H)) {
bd1 while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2])))
f6e dq.pop_back(), --len;
eee while (len > 1 && H[i].out(inter(dq[0], dq[1])))
62a dq.pop_front(), --len;
b06 if (len > 0 && fabs1(H[i].pq.cross(dq[len-1].pq)) < eps)
{
02c if (H[i].pq.dot(dq[len-1].pq) < 0.0) return vector<P>
>();
da8 if (H[i].out(dq[len-1].p) dq.pop_back(), --len;
777 else continue;
bb8 }
d79 dq.push_back(H[i]), ++len;
80a }
5b5 while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2])))
b40 dq.pop_back(), --len;
ac7 while (len > 2 && dq[len-1].out(inter(dq[0], dq[1])))
24b dq.pop_front(), --len;
2de if (len < 3) return vector<P>();
3cf vector<P> ret(len);
eed rep(i, 0, len-1) ret[i] = inter(dq[i], dq[i+1]);
80e ret.back() = inter(dq[len-1], dq[0]);
00d return ret;
99c }
```

8.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

	3058c3, 6 lines
--	-----------------

```
f9c template<class V, class L>
8b5 double signedPolyVolume(const V& p, const L& trilst) {
75c double v = 0;
828 for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c
]);
27c return v / 6;
305 }
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

	a94b7c, 31 lines
--	------------------

```
f10 template<class T> struct Point3D {
144 typedef Point3D P;
6c9 T x, y, z;
824 explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {
}
e9d bool operator<(P p) const {
return tie(x, y, z) < tie(p.x, p.y, p.z); }
082 bool operator==(P p) const {
1c4 return tie(x, y, z) == tie(p.x, p.y, p.z); }
f67 P operator+(P p) const { return P(x+p.x, y+p.y, z+p.z); }
f28 P operator-(P p) const { return P(x-p.x, y-p.y, z-p.z); }
9dc P operator*(T d) const { return P(x*d, y*d, z*d); }
b6d P operator/(T d) const { return P(x/d, y/d, z/d); }
9d1 T dot(P p) const { return x*p.x + y*p.y + z*p.z; }
404 P cross(P p) const {
5d1 return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
ba7 }
4b3 T dist2() const { return x*x + y*y + z*z; }
f6b double dist() const { return sqrt((double)dist2()); }
//Azimuthal angle (longitude) to the x-axis in interval [-pi,
pi]
e19 double phi() const { return atan2(y, x); }
//Zenith angle (latitude) to the z-axis in interval [0,
```

```
7e4 double theta() const { return atan2(sqrt(x*x+y*y),z); }
537 P unit() const { return *this/(T)dist(); } //makes dist()
=1
//returns unit vector normal to *this and p
098 P normal(P p) const { return cross(p).unit(); }
//returns point rotated 'angle' radians ccw around axis
720 P rotate(double angle, P axis) const {
dff double s = sin(angle), c = cos(angle); P u = axis.unit()
;
15c return u.dot(u)*(1-c) + (*this)*c - cross(u)*s;
2e2 }
a94 };
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $O(n^2)$

"Point3D.h"	5b45fc, 49 lines
-------------	------------------

```
b8e typedef Point3D<double> P3;

6aa struct PR {
cc2 void ins(int x) { (a == -1 ? a : b) = x; }
e28 void rem(int x) { (a == x ? a : b) = -1; }
531 int cnt() { return (a != -1) + (b != -1); }
5f7 int a, b;
9a9 };

538 struct F { P3 q; int a, b, c; };

7d6 vector<F> hull3d(const vector<P3>& A) {
1d7 assert(sz(A) >= 4);
39c vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
39d #define E(x,y) E[f.x][f.y]
6ee vector<F> FS;
946 auto mf = [&](int i, int j, int k, int l) {
47e P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
60a if (q.dot(A[l]) > q.dot(A[i]))
d64 q = q * -1;
ed7 F f(q, i, j, k);
dd2 E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
d2c FS.push_back(f);
f13 };
411 rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
489 mf(i, j, k, 6 - i - j - k);

42c rep(i,4,sz(A)) {
b33 rep(j,0,sz(FS)) {
77d F f = FS[j];
c1b if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
d54 E(a,b).rem(f.c);
6ed E(a,c).rem(f.b);
538 E(b,c).rem(f.a);
2eb swap(FS[j--], FS.back());
324 FS.pop_back();
40e }
661 }
47a int nw = sz(FS);
930 rep(j,0,nw) {
5d8 F f = FS[j];
460 #define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.
c);
ccc C(a, b, c); C(a, c, b); C(b, c, a);
9bd }
c8c }
299 for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
362 A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
7f1 return FS;
5b4 };
```


sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius r between the points with azimuthal angles (longitude) f_1 (ϕ_1) and f_2 (ϕ_2) from x axis and zenith angles (latitude) t_1 (θ_1) and t_2 (θ_2) from z axis ($0 =$ north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. $dx \cdot radius$ is then the difference between the two points in the x direction and $d \cdot radius$ is the total distance between the points.

	611f07, 8 lines
c5f	double sphericalDistance(double f1, double t1,
86b	double f2, double t2, double radius) {
2b5	double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
aa0	double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
6da	double dz = cos(t2) - cos(t1);
819	double d = sqrt(dx*dx + dy*dy + dz*dz);
e1b	return radius*2*asin(d/2);
611	}

Strings (9)

KMP.h

Description: $pi[x]$ computes the length of the longest prefix of s that ends at x , other than $s[0...x]$ itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$	d4375c, 16 lines
-------------------------------	------------------

132	vi pi(const string& s) {
ada	vi p(sz(s));
049	rep(i,1,sz(s)) {
c04	int g = p[i-1];
f6d	while (g && s[i] != s[g]) g = p[g-1];
21a	p[i] = g + (s[i] == s[g]);
6c1	}
63e	return p;
9cb	}

Zfunc.h

Description: $z[i]$ computes the length of the longest common prefix of $s[i:]$ and s , except $z[0] = 0$. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

	ee09e2, 12 lines
44d	vi Z(const string& S) {
cf9	vi z(sz(S));
661	int l = -1, r = -1;
4fa	rep(i,1,sz(S)) {
fc3	z[i] = i >= r ? 0 : min(r - i, z[i - l]);
145	while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
d97	z[i]++;
18e	if (i + z[i] > r)
0d1	l = i, r = i + z[i];
b90	}
cc0	return z;
ee0	}

Manacher.h

Description: For each position in a string, computes $p[0][i] =$ half length of longest even palindrome around pos i , $p[1][i] =$ longest odd (half rounded down).

Time: $\mathcal{O}(N)$	e7ad79, 13 lines
-------------------------------	------------------

fc1	array<vi, 2> manacher(const string& s) {
-----	--

f03	int n = sz(s);
daf	array<vi,2> p = {vi(n+1), vi(n)};
4d1	rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
510	int t = r-i+!z;
2a5	if (i<r) p[z][i] = min(t, p[z][l+t]);
361	int L = i-p[z][i], R = i+p[z][i]-!z;
508	while (L>=1 && R+1<n && s[L-1] == s[R+1])
c79	p[z][i]++, L--, R++;
168	if (R>r) l=L, r=R;
21a	}
4ae	return p;
e7a	}

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());

Time: $\mathcal{O}(N)$

	d07a42, 8 lines
5fa	int minRotation(string s) {
e7c	int a=0, N=sz(s); s += s;
5ca	rep(b,0,N) rep(k,0,N) {
f65	if (a+k == b s[a+k] < s[b+k]) {b += max(0, k-1);
	break;}
20f	if (s[a+k] > s[b+k]) { a = b; break; }
b2e	}
5fa	return a;
d07	}

SuffixArray.h

Description: Builds suffix array for a string. $sa[i]$ is the starting index of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n + 1$, and $sa[0] = n$. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: $lcp[i] = lcp(sa[i], sa[i-1])$, $lcp[0] = 0$. The input string must not contain any nul chars.

Time: $\mathcal{O}(n \log n)$	635552, 22 lines
--------------------------------------	------------------

3f4	struct SuffixArray {
1rf	vi sa, lcp;
12e	SuffixArray(string s, int lim=256) { // or vector<int>
ca6	s.push_back(0); int n = sz(s), k = 0, a, b;
73d	vi x(all(s)), y(n), ws(max(n, lim));
2c6	sa = lcp = y, iota(all(sa), 0);
2b6	for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p
	) {
69d	p = j, iota(all(y), n - j);
35f	rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
4e8	fill(all(ws), 0);
058	rep(i,0,n) ws[x[i]]++;
8a7	rep(i,1,lim) ws[i] += ws[i - 1];
6e5	for (int i = n; i--;) sa[--ws[x[i]]] = y[i];
0ba	swap(x, y), p = 1, x[sa[0]] = 0;
0f8	rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
967	(y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p
	++;
7ff	}
089	for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
bb4	for (k && k--, j = sa[x[i] - 1];
2db	s[i + k] == s[j + k]; k++);
c24	}
635	};

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices $[l, r]$ into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining $[l, r]$ substrings. The root is 0 (has $l = -1, r = 0$), non-existent children are -1. To get a complete tree, append a dummy symbol - otherwise it may contain an incomplete path (still useful for substring matching, though).

Time: $\mathcal{O}(26N)$	aae0b8, 50 lines
---------------------------------	------------------

3a1	struct SuffixTree {
749	enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
e2a	int toi(char c) { return c - 'a'; }
c01	string a; // v = cur node, q = cur position
b1f	int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;
557	void ukkadd(int i, int c) { suff:
a82	if (r[v]<=q) {
f7d	if (t[v][c]==-1) { t[v][c]=m; l[m]=i;
810	p[m++]=v; v=s[v]; q=r[v]; goto suff; }
54a	v=t[v][c]; q=l[v];
6b5	}
8cb	if (q==-1 c==toi(a[q])) q++; else {
141	l[m+1]=i; p[m+1]=m; l[m]=l[v]; r[m]=q;
694	p[m]=p[v]; t[m][c]=m+1; t[m][toi(a[q])]=v;
d13	l[v]=q; p[v]=m; t[p[m]][toi(a[l[m])]]=m;
d6d	v=s[p[m]]; q=l[m];
afa	while (q<r[m]) { v=t[v][toi(a[q])]; q+=r[v]-l[v]; }
8b3	if (q==r[m]) s[m]=v; else s[m]=m+2;
3ea	q=r[v]-(q-r[m]); m+=2; goto suff;
451	}
0b7	}

d44	SuffixTree(string a) : a(a) {
88d	fill(r,r+N,sz(a));
cb2	memset(s, 0, sizeof s);
ab0	memset(t, -1, sizeof t);
0a9	fill(t[1],t[1]+ALPHA,0);
3b1	s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] =
	0;
372	rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
e6a	}

	// example: find longest common substring (uses ALPHA = 28)
66b	pii best;
66d	int lcs(int node, int i1, int i2, int olen) {
dc2	if (l[node] <= i1 && i1 < r[node]) return l;
bb9	if (l[node] <= i2 && i2 < r[node]) return 2;
088	int mask = 0, len = node ? olen + (r[node] - l[node]) :
	0;
361	rep(c,0,ALPHA) if (t[node][c] != -1)
7c5	mask = lcs(t[node][c], i1, i2, len);
f72	if (mask == 3)
f77	best = max(best, {len, r[node] - len});
2dd	return mask;
526	}
e2e	static pii LCS(string s, string t) {
a84	SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2))
	;
fb6	st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
ccb	return st.best;
9dc	}
aae	};

SuffixAutomaton.h

Description: Suffix Automaton, a compressed form of all substrings of a given string.

Time: construction takes $\mathcal{O}(N \log K)$, where $N =$ size of the string and $K =$ size of the alphabet. getDiffStrings() is $\mathcal{O}(N)$.

	8c2be7, 42 lines
92e	struct SuffixAutomaton {
f16	struct State {
a8a	int len, link;
780	map<char, int> next;
d05	};
b10	int m;
2f9	vector<State> st;

```
78b SuffixAutomaton(string s): m(1), st(2 * sz(s)) {
a72 st[0].len = 0, st[0].link = -1;
4c8 int last = 0;
582 for (char c : s) {
edc int cur = m++, p = last;
288 st[cur].len = st[last].len + 1;
595 while (p != -1 && !st[p].next.count(c)) {
e41 st[p].next[c] = cur;
8a3 p = st[p].link;
1e0 }
e27 if (p == -1) st[cur].link = 0;
94a else {
1a8 int q = st[p].next[c];
7c2 if (st[p].len + 1 == st[q].len) st[cur].link = q;
3b8 else {
395 int clone = m++;
2ad st[clone].len = st[p].len + 1;
bbc st[clone].next = st[q].next;
601 st[clone].link = st[q].link;
141 while (p != -1 && st[p].next[c] == q) {
30d st[p].next[c] = clone;
ac2 p = st[p].link;
a92 }
143 st[q].link = st[cur].link = clone;
0d1 }
f37 }
b38 last = cur;
df2 }
570 }
fbb ll getDiffStrings() {
a94 ll tot = 0;
9c8 rep(i, 1, m) tot += st[i].len - st[st[i].link].len;
e7c return tot;
643 }
8c2 };
```

Hashing.h
Description: Self-explanatory methods for string hashing. 2d2a677, 44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
41d typedef uint64_t ull;
953 struct H {
80c ull x; H(ull x=0) : x(x) {}
1f9 H operator+(H o) { return x + o.x + (x + o.x < x); }
98c H operator-(H o) { return *this + ~o.x; }
df4 H operator*(H o) { auto m = (__uint128_t)x * o.x;
4ef return H((ull)m) + (ull)(m >> 64); }
f17 ull get() const { return x + !~x; }
f8f bool operator==(H o) const { return get() == o.get(); }
442 bool operator<(H o) const { return get() < o.get(); }
40d };
274 static const H C = (11)1e11+3; // (order ~ 3e9; random also
ok)
```

```
ff1 struct HashInterval {
558 vector<H> ha, pw;
76d HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
ed3 pw[0] = 1;
77e rep(i,0,sz(str))
aad ha[i+1] = ha[i] * C + str[i],
7cc pw[i+1] = pw[i] * C;
474 }
90c H hashInterval(int a, int b) { // hash [a, b)
143 return ha[b] - ha[a] * pw[b - a];
c82 }
```

```
91d };

690 vector<H> getHashes(string& str, int length) {
6b6 if (sz(str) < length) return {};
355 H h = 0, pw = 1;
22c rep(i,0,length)
a4e h = h * C + str[i], pw = pw * C;
707 vector<H> ret = {h};
e1b rep(i,length,sz(str)) {
54d ret.push_back(h = h * C + str[i] - pw * str[i-length]);
0f0 }
431 return ret;
2f2 }

2d2 H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return
h};
```

AhoCorasick.h
Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$. f35677, 66 lines

```
51f struct AhoCorasick {
ba2 enum {alpha = 26, first = 'A'}; // change this!
b51 struct Node {
// (nmatches is optional)
724 int back, next[alpha], start = -1, end = -1, nmatches =
0;
c32 Node(int v) { memset(next, v, sizeof(next)); }
9e9 };
99b vector<Node> N;
b8e vi backp;
abd void insert(string& s, int j) {
77a assert(!s.empty());
777 int n = 0;
503 for (char c : s) {
77a int& m = N[n].next[c - first];
eeb if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
d6c else n = m;
4fc }
66f if (N[n].end == -1) N[n].start = j;
695 backp.push_back(N[n].end);
c0e N[n].end = j;
702 N[n].nmatches++;
10d }
bc0 AhoCorasick(vector<string>& pat) : N(1, -1) {
32a rep(i,0,sz(pat)) insert(pat[i], i);
514 N[0].back = sz(N);
704 N.emplace_back(0);

8b6 queue<int> q;
d64 for (q.push(0); !q.empty(); q.pop()) {
e8c int n = q.front(), prev = N[n].back;
eab rep(i,0,alpha) {
644 int &ed = N[n].next[i], y = N[prev].next[i];
fe3 if (ed == -1) ed = y;
040 else {
0ea N[ed].back = y;
1c3 (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
= N[y].end;
912 N[ed].nmatches += N[y].nmatches;
3ab q.push(ed);
021 }
```

```
4d5 }
ddd }
22d }
725 }
034 vi find(string word) {
8fa int n = 0;
af9 vi res; // ll count = 0;
7a9 for (char c : word) {
a99 n = N[n].next[c - first];
8d0 res.push_back(N[n].end);
// count += N[n].nmatches;

a23 }
85d return res;
33c }
32e vector<vi> findAll(vector<string>& pat, string word) {
801 vi r = find(word);
7dd vector<vi> res(sz(word));
910 rep(i,0,sz(word)) {
0f3 int ind = r[i];
0ec while (ind != -1) {
079 res[i - sz(pat[ind]) + 1].push_back(ind);
328 ind = backp[ind];
2f7 }
675 }
cbc return res;
e9b }
f35 };
```

Various (10)

10.1 Intervals

IntervalContainer.h
Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive). Time: $\mathcal{O}(\log N)$ edce47, 23 lines

```
d91 set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
905 if (L == R) return is.end();
117 auto it = is.lower_bound({L, R}), before = it;
b01 while (it != is.end() && it->first <= R) {
ba1 R = max(R, it->second);
a98 before = it = is.erase(it);
381 }
6d8 if (it != is.begin() && (--it)->second >= L) {
7d7 L = min(L, it->first);
d2f R = max(R, it->second);
8ea is.erase(it);
578 }
72f return is.insert(before, {L,R});
d57 }
```

```
154 void removeInterval(set<pii>& is, int L, int R) {
969 if (L == R) return;
f20 auto it = addInterval(is, L, R);
51c auto r2 = it->second;
d09 if (it->first == L) is.erase(it);
c1d else (int&)it->second = L;
b4d if (R != r2) is.emplace(R, r2);
edc }
```

IntervalCover.h
Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty). Time: $\mathcal{O}(N \log N)$ 9e9d8d, 19 lines

4fc	template<class T>
68e	vi cover(pair<T, T> G, vector<pair<T, T>> I) {
1a8	vi S(sz(I)), R;
fa5	iota(all(S), 0);
0e2	sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
a16	T cur = G.first;
4d4	int at = 0;
7ca	while (cur < G.second) { // (A)
3ef	pair<T, int> mx = make_pair(cur, -1);
bfc	while (at < sz(I) && I[S[at]].first <= cur) {
201	mx = max(mx, make_pair(I[S[at]].second, S[at]));
9b4	at++;
470	}
f1e	if (mx.second == -1) return {};
206	cur = mx.first;
932	R.push_back(mx.second);
cd0	}
c6a	return R;
9e9	}

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

Usage: constantIntervals(0, sz(v), [&](int x){return v[x];},
 [&](int lo, int hi, T val){...});

Time: $\mathcal{O}(k \log \frac{n}{k})$

	753a4c, 19 lines
fb4	template<class F, class G, class T>
4f5	void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
801	if (p == q) return ;
ef5	if (from == to) {
570	g(i, to, p);
a2e	i = to; p = q;
143	} else {
e7d	int mid = (from + to) >> 1;
a8d	rec(from, mid, f, g, i, p, f(mid));
1ff	rec(mid+1, to, f, g, i, p, q);
5b6	}
69b	}
3eb	template<class F, class G>
8dd	void constantIntervals(int from, int to, F f, G g) {
7f4	if (to <= from) return ;
e84	int i = from; auto p = f(i), q = f(to-1);
587	rec(from, to-1, f, g, i, p, q);
d2f	g(i, to, q);
753	}

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in [a,b] that maximizes $f(i)$, assuming that $f(a) < \dots < f(i) \geq \dots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).

Usage: int ind = ternSearch(0,n-1,[&](int i){return a[i];});
Time: $\mathcal{O}(\log(b-a))$

	9155b4, 11 lines
044	template<class F>
038	int ternSearch(int a, int b, F f) {
845	assert(a <= b);
f3c	while (b - a >= 5) {
7d4	int mid = (a + b) / 2;
850	if (f(mid) < f(mid+1)) a = mid; // (A)
c62	else b = mid+1;
17d	}
09c	rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
c06	return a;
915	}

LIS.h

Description: Compute indices for the longest increasing subsequence.

Time: $\mathcal{O}(N \log N)$

	2932a0, 17 lines
8d3	template<class I> vi lis(const vector<I>& S) {
be1	if (S.empty()) return {};
67f	vi prev(sz(S));
c1c	typedef pair<I, int> p;
47f	vector<p> res;
5dc	rep(i,0,sz(S)) {
	// change 0 -> i for longest non-decreasing subsequence
f6e	auto it = lower_bound(all(res), p[S[i], 0]);
b92	if (it == res.end()) res.emplace_back(), it = res.end()
	-1;
26a	*it = {S[i], i};
e3b	prev[i] = it == res.begin() ? 0 : (it-1)->second;
f2e	}
b33	int L = sz(res), cur = res.back().second;
3f0	vi ans(L);
ffa	while (L--) ans[L] = cur, cur = prev[cur];
459	return ans;
293	}

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.

Time: $\mathcal{O}(N \max(w_i))$

	b20ccc, 16 lines
4d3	int knapsack(vi w, int t) {
cca	int a = 0, b = 0, x;
ba5	while (b < sz(w) && a + w[b] <= t) a += w[b++];
3f6	if (b == sz(w)) return a;
e2b	int m = *max_element(all(w));
11f	vi u, v(2*m, -1);
7d8	v[a+m-t] = b;
682	rep(i,b,sz(w)) {
c83	u = v;
e89	rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
51a	for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
5e6	v[x-w[j]] = max(v[x-w[j]], j);
d2b	}
700	for (a = t; v[a+m-t] < 0; a--);
e4d	return a;
b20	}

SumOverSubsets.h

Description: Given an array a with 2^n elements, computes $f(x)$ for all $i = 0, 1, 2, \dots, 2^n - 1$. Here, $f(x)$ represents the sum of values in array a for all subsets i of x. That is, $f(x) = \sum_{i \subseteq x} a[i]$.

Time: $\mathcal{O}(2^n n)$

	79aab5, 7 lines
20a	vector<ll> sos(vector<ll> a) {
706	int n = __lg(sz(a));
c6c	rep(i, 0, n) rep(x, 0, 1 << n) {
8c2	if (x & (1 << i)) a[x] += a[x ^ (1 << i)]; // ~x & (1 << i) for superset
675	}
d17	return a;
79a	}

10.3 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j - 1]$ and $p[i + 1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i) \leq k < hi(i)} (f(i, k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R - 1$.

Time: $\mathcal{O}((N + (hi - lo)) \log N)$

	d38d2b, 18 lines
242	struct DP { // Modify at will:
ce5	int lo(int ind) { return 0; }
f74	int hi(int ind) { return ind; }
29e	ll f(int ind, int k) { return dp[ind][k]; }
ff9	void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
275	void rec(int L, int R, int LO, int HI) {
d26	if (L >= R) return ;
13d	int mid = (L + R) >> 1;
81b	pair<ll, int> best (LLONG_MAX, LO);
c3f	rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
7ff	best = min(best, make_pair(f(mid, k), k));
1a4	store(mid, best.second, best.first);
499	rec(L, mid, LO, best.second+1);
1eb	rec(mid+1, R, best.second, HI);
116	}
97a	void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
d38	};

10.4 Debugging tricks

- signal(SIGSEGV, [](int) { _Exit(0); });
converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). _GLIBCXX_DEBUG failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).

- feenableexcept (29); kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

__builtin_ia32_ldmxcsr (40896); disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- x & -x is the least bit in x.
- **for** (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
- c = x&-x, r = x+c; ((r^x) >> 2)/c | r is the next number after x with the same number of bits set.
- rep(b,0,K) rep(i,0,(1 << K))
 if (i & 1 << b) D[i] += D[i^(1 << b)];
computes all sums of subsets.

10.5.2 Pragmas

- **#pragma** GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.
- **#pragma** GCC target ("avx2") can double performance of vectorized code, but causes crashes on old machines.
- **#pragma** GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to a (mod b) in the range $[0, 2b)$.

	751a02, 8 lines
f4c	typedef unsigned long long ull;
a7a	struct FastMod {
a51	ull b, m;
551	FastMod(ull b) : b(b), m(-1ULL / b) {}
010	ull reduce(ull a) { <i>// a % b + (0 or b)</i>
c7e	return a - (ull)((__uint128_t(m) * a) >> 64) * b;
03d	}
751	};

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.

Usage: ./a.out < input.txt

Time: About 5x as fast as cin/scanf.

	7b3c70, 17 lines
c30	inline char gc() { <i>// like getchar()</i>
b53	static char buf[1 << 16];
0c0	static size_t bc, be;
62a	if (bc >= be) {
c51	buf[0] = 0, bc = 0;
bba	be = fread(buf, 1, sizeof (buf), stdin);
e9a	}
973	return buf[bc++]; <i>// returns 0 on EOF</i>
026	}
b36	int readInt() {
b81	int a, c;
d55	while ((a = gc()) < 40);
bc5	if (a == '-') return -readInt();
e7b	while ((c = gc()) >= 48) a = a * 10 + c - 480;
5eb	return a - 48;
7b3	}

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

	745db2, 8 lines
<i>// Either globally or in a single class:</i>	
2b9	static char buf[450 << 20];
73a	void* operator new (size_t s) {
3d5	static size_t i = sizeof buf;
c17	assert (s < i);
e69	return (void*)&buf[i -= s];
0c4	}
745	void operator delete (void*) {}

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

	2dd6c9, 10 lines
0ca	template < class T> struct ptr {
d7b	unsigned ind;

953	ptr(T* p = 0) : ind(p ? unsigned ((char*)p - buf) : 0) {
347	assert (ind < sizeof buf);
bda	}
36a	T& operator* () const { return *(T*)(buf + ind); }
c82	T* operator-> () const { return &* this ; }
dd2	T& operator[] (int a) const { return (&* this)[a]; }
881	explicit operator bool () const { return ind; }
2dd	};

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.

	bb66d4, 14 lines
30c	char buf[450 << 20] alignas(16);
fbe	size_t buf_ind = sizeof buf;
c23	template < class T> struct small {
2c8	typedef T value_type;
bea	small() {}
a4e	template < class U> small(const U&) {}
d50	T* allocate (size_t n) {
24f	buf_ind -= n * sizeof (T);
95c	buf_ind &= 0 - alignof(T);
f6f	return (T*)(buf + buf_ind);
16a	}
92a	void deallocate (T*, size_t) {}
bb6	};

SIMD.h

Description: Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern `"_mm(256)?name.(si(128|256)|epi(8|16|32|64)|pd|ps)"`. Not all are described here; grep for `_mm_` in `/usr/lib/gcc/*/4.9/include/` for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and `#define __SSE__` and `__MMX__` before including it. For aligned memory use `_mm_malloc(size, 32)` or `int buf[N] alignas(32)`, but prefer `loadu/storeu`.

	551b82, 43 lines
ee8	#pragma GCC target ("avx2") <i>// or sse4.1</i>
ee5	#include "emmintrin.h"
b99	typedef __m256i mi;
c6d	#define L(x) _mm256_loadu_si256((mi*)&(x))
<i>// High-level/specific methods:</i>	
<i>// load(u)?_si256, store(u)?_si256, setzero_si256, _mm_malloc</i>	
<i>// blendv_(epi8 ps pd) (z?y:x), movemask_epi8 (hibits of bytes)</i>	
<i>// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x</i>	
<i>// sad_epu8: sum of absolute differences of u8, outputs 4xi64</i>	
<i>// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15</i>	
<i>// madd_epi16: dot product of signed i16's, outputs 8xi32</i>	
<i>// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)</i>	
<i>// permute2f128_si256(x,x,1) swaps 128-bit lanes</i>	
<i>// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane</i>	
<i>// shuffle_epi8(x, y) takes a vector instead of an imm</i>	
<i>// Methods that work with most data types (append e.g. _epi32):</i>	
<i>// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,</i>	
<i>// andnot, abs, min, max, sign(i,x), cmp(gt eq), unpack(lo hi)</i>	
597	int sumi32(mi m) { union { int v[8]; mi m;} u; u.m = m;
22d	int ret = 0; rep(i,0,8) ret += u.v[i]; return ret; }
482	mi zero() { return _mm256_setzero_si256(); }
388	mi one() { return _mm256_set1_epi32(-1); }
66c	bool all_zero(mi m) { return _mm256_testz_si256(m, m); }
f14	bool all_one(mi m) { return _mm256_testc_si256(m, one()); }
732	ll example_filteredDotProduct(int n, short* a, short* b) {

a0b	int i = 0; ll r = 0;
d95	mi zero = _mm256_setzero_si256(), acc = zero;
4fd	while (i + 16 <= n) {
f51	mi va = L(a[i]), vb = L(b[i]); i += 16;
957	va = _mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
973	mi vp = _mm256_madd_epi16(va, vb);
784	acc = _mm256_add_epi64(_mm256_unpacklo_epi32(vp, zero),
25a	_mm256_add_epi64(acc, _mm256_unpackhi_epi32(vp, zero))
	);
8de	}
b89	union {ll v[4]; mi m;} u; u.m = acc; rep(i,0,4) r += u.v[i];
844	for (;i<n;++i) if (a[i] < b[i]) r += a[i]*b[i]; <i>// <-equiv</i>
4a4	return r;
551	}

Techniques (A)

techniques.txt

159 lines

Recursion

Divide and conquer

Finding interesting points in N log N

Algorithm analysis

Master theorem

Amortized time complexity

Greedy algorithm

Scheduling

Max contiguous subvector sum

Invariants

Huffman encoding

Graph theory

Dynamic graphs (extra book-keeping)

Breadth first search

Depth first search

* Normal trees / DFS trees

Dijkstra’s algorithm

MST: Prim’s algorithm

Bellman-Ford

Konig’s theorem and vertex cover

Min-cost max flow

Lovasz toggle

Matrix tree theorem

Maximal matching, general graphs

Hopcroft-Karp

Hall’s marriage theorem

Graphical sequences

Floyd-Warshall

Euler cycles

Flow networks

* Augmenting paths

* Edmonds-Karp

Bipartite matching

Min. path cover

Topological sorting

Strongly connected components

2-SAT

Cut vertices, cut-edges and biconnected components

Edge coloring

* Trees

Vertex coloring

* Bipartite graphs (=> trees)

* 3^n (special case of set cover)

Diameter and centroid

K’t h shortest path

Shortest cycle

Dynamic programming

Knapsack

Coin change

Longest common subsequence

Longest increasing subsequence

Number of paths in a dag

Shortest path in a dag

Dynprog over intervals

Dynprog over subsets

Dynprog over probabilities

Dynprog over trees

3^n set cover

Divide and conquer

Knuth optimization

Convex hull optimizations

RMQ (sparse table a.k.a 2^k-jumps)

Bitonic cycle

Log partitioning (loop over most restricted)

Combinatorics

Computation of binomial coefficients

Pigeon-hole principle

Inclusion/exclusion

Catalan number

Pick’s theorem

Number theory

Integer parts

Divisibility

Euclidean algorithm

Modular arithmetic

* Modular multiplication

* Modular inverses

* Modular exponentiation by squaring

Chinese remainder theorem

Fermat’s little theorem

Euler’s theorem

Phi function

Frobenius number

Quadratic reciprocity

Pollard-Rho

Miller-Rabin

Hensel lifting

Vieta root jumping

Game theory

Combinatorial games

Game trees

Mini-max

Nim

Games on graphs

Games on graphs with loops

Grundy numbers

Bipartite games without repetition

General games without repetition

Alpha-beta pruning

Probability theory

Optimization

Binary search

Ternary search

Unimodality and convex functions

Binary search on derivative

Numerical methods

Numeric integration

Newton’s method

Root-finding with binary/ternary search

Golden section search

Matrices

Gaussian elimination

Exponentiation by squaring

Sorting

Radix sort

Geometry

Coordinates and vectors

* Cross product

* Scalar product

Convex hull

Polygon cut

Closest pair

Coordinate-compression

Quadtrees

KD-trees

All segment-segment intersection

Sweeping

Discretization (convert to events and sweep)

Angle sweeping

Line sweeping

Discrete second derivatives

Strings

Longest common substring

Palindrome subsequences

Knuth-Morris-Pratt

Tries

Rolling polynomial hashes

Suffix array

Suffix tree

Aho-Corasick

Manacher’s algorithm

Letter position lists

Combinatorial search

Meet in the middle

Brute-force with pruning

Best-first (A*)

Bidirectional search

Iterative deepening DFS / A*

Data structures

LCA (2^k-jumps in trees in general)

Pull/push-technique on trees

Heavy-light decomposition

Centroid decomposition

Lazy propagation

Self-balancing trees

Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)

Monotone queues / monotone stacks / sliding queues

Sliding queue using 2 stacks

Persistent segment tree